

บทที่ 2

ทฤษฎีและหลักการ

2.1 เว็บเซอร์วิส (Web Service)

2.1.1 การพัฒนาของเว็บ

การพัฒนาทางเทคโนโลยีใด ๆ ล้วนแล้วแต่มีพัฒนาการเป็นยุคสมัย แต่ละยุคสมัยมีการใช้เทคโนโลยีที่มีจุดสำคัญของการพัฒนาที่เด่นชัด การพัฒนาเทคโนโลยีการประยุกต์ใช้งานบนเว็บก็เช่นเดียวกัน

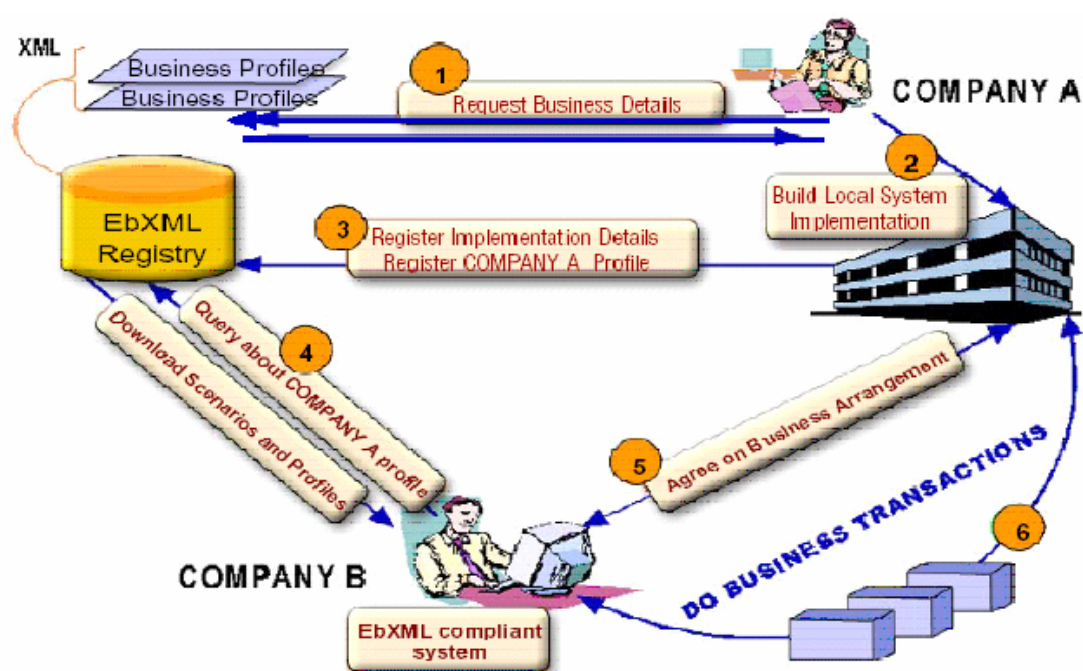
2.1.1.1 สเตติกเว็บเพจ (Static Web Page)

เป็นยุคที่เริ่มต้นการใช้งานเว็บมีการวางข้อมูลตามมาตรฐานเฮททีเอ็มแอล (HTML) ไว้เป็นไฟล์บนเซิร์ฟเวอร์ ผู้ใช้งานใช้เบราว์เซอร์ (browser) เรียกข้อมูลด้วยโปรโตคอลเอชทีทีพี การเรียกใช้ข้อมูลจะได้ข้อมูลที่เก็บไว้บนเซิร์ฟเวอร์ โดยลักษณะของข้อมูลยังเป็นข้อมูลแบบสเตติก คือเก็บไว้เพื่อรอการเรียกใช้ บนฝั่งของเซิร์ฟเวอร์เองยังไม่มีโปรแกรมพิเศษใด ๆ เพียงแต่มีโปรแกรมที่รับการติดต่อด้วยเอชทีทีพีเท่านั้น ข้อมูลที่เรียกใช้ได้รับมาเป็นไฟล์ โดยทางฝั่งไคลเอนต์จะนำไปแสดงผลตามข้อมูลที่ปรากฏในเท็กซ์ (text) ตามมาตรฐานในยุคที่หนึ่งนี้ทำให้มีการเก็บข้อมูลบนเซิร์ฟเวอร์เพื่อเป็นจุดที่ทำให้ผู้ใช้งานเรียกเข้าใช้ได้ง่าย การพัฒนาระบบบริการข้อมูลแบบเว็บเพจ (web page) หรือ โฮมเพจ (home page) จึงได้รับความสนใจ ทำให้มีผู้สร้างเว็บเพจและผู้ใช้ผ่านเบราว์เซอร์เป็นจำนวนมาก

2.1.1.2 ไดนามิกเว็บเพจ (Dynamic Web Page)

มีการพัฒนาโปรแกรมทางฝั่งเซิร์ฟเวอร์ เพื่อให้เซิร์ฟเวอร์ทำงานตามคำเรียกขอมมาจากทางไคลเอนต์ผ่านทางโปรโตคอลเอชทีทีพี หรือการเรียกของบนเว็บ มีการเชื่อมโยงกับโปรแกรมเฉพาะ เช่นเมื่อผู้เรียกใช้คลิกบนเบราว์เซอร์ ทำให้เกิดการเรียกขอมที่เซิร์ฟเวอร์ เซิร์ฟเวอร์จะทำงานตอบสนอง โดยอาจจะไปรันโปรแกรม หรือเรียกโปรแกรมอื่นที่ทางฝั่งเซิร์ฟเวอร์เตรียมไว้ ในยุคสมัยนี้มีการสร้างระบบการเรียกเข้าหาโปรแกรมที่เรียกว่า ซีจีไอ หรือ คอมมอนเกตเวย์อินเทอร์เฟส (CGI-Common Gateway Interface) การเขียนโปรแกรมมีเทคนิควิธีการพิเศษที่ทำให้เข้าถึงฐานข้อมูล มีการพัฒนาเครื่องมือการพัฒนา และภาษาสำหรับพัฒนาเพื่อทำให้เรียกข้อมูลจากฐานข้อมูลหรือเรียกเพิ่มข้อมูลอื่นเพื่อส่งต่อไปให้ผู้เรียกใช้ ภาษาที่นิยมใช้ในการพัฒนามีภาษาซี (C Language) ภาษาเพิร์ล (PERL Language) และเครื่องมือพิเศษที่ใช้เชื่อมโยงกับฐานข้อมูลที่ผู้ผลิตซอฟต์แวร์ทางฐานข้อมูลดำเนินการให้ ตลอดจนมีการใช้โปรแกรมทางด้านไคลเอนต์ที่เริ่มซับซ้อนขึ้น

การที่เรียกว่าไดนามิกเว็บเพจก็ด้วยเหตุผลที่ว่าข้อมูลที่เรียกดูจะแปรเปลี่ยนตามการเรียกขอ การปรับเปลี่ยนข้อมูลกระทำตามโปรแกรมที่สร้างขึ้น สามารถคำนวณหรือประมาณผลข้อมูลได้ เช่น การดูข้อมูลสรุปของแต่ละวัน ดังนั้นการเรียกดูข้อมูลจะได้ผลที่ทันสมัยได้ ระบบเซิร์ฟเวอร์จึงเชื่อมโยงกับฐานข้อมูลหรือระบบงานอื่น แต่เมื่อจะส่งข้อมูลไปยังผู้ใช้ที่เรียกขอเข้ามา จะแปลงให้อยู่ในรูปแบบของเอชทีเอ็มแอลก่อนเพื่อให้เบราว์เซอร์แสดงผลได้



รูปที่ 2-1 ลักษณะของเว็บเซอร์วิส

2.1.1.3 บริการบนเว็บ (Web Service)

การพัฒนาเว็บมาจนถึงปัจจุบันได้ก้าวหน้าขึ้นไปอีกมาก จุดสำคัญอยู่ที่ความต้องการรองรับการใช้งานที่มีความต้องการสูงขึ้น โดยเฉพาะการดำเนินการทางธุรกิจการค้าต่างๆ ที่เพิ่มมากขึ้นบนเครือข่าย ลักษณะงานที่ต้องการมีลักษณะการทำงานร่วมกันระหว่างองค์กร (Interoperability) โดยให้โปรแกรมประยุกต์ขององค์กรหนึ่งส่งคำขอผ่านเครือข่ายอินเทอร์เน็ตด้วยโปรโตคอลเอชทีทีพี ไปยังเว็บบริการของอีกองค์กรหนึ่ง มีการโต้ตอบเพื่อรับส่งข้อมูลระหว่างกันแบบอัตโนมัติได้ การเรียกใช้บริการต่างๆ มีลักษณะการเรียกใช้บริการ โดยมีตัวกลางช่วยและเชื่อมโยงเข้ากับเว็บเซิร์ฟเวอร์ การแลกเปลี่ยนข้อมูลตามคำเรียกขอใช้มาตรฐานเอ็กซ์เอ็มแอล หรือเอ็กซ์เทนซิเบิลมาร์กอัปแลงเกวจ (XML-eXtensible Mark Up Language) ข้อมูลที่เป็นเอ็กซ์เอ็มแอลจะถูกส่งผ่านมายาได้การเรียกขอเอชทีทีพี การบริการบนเว็บจึงเป็นการก้าวเข้ามาในระบบที่ทำให้เกิดการแลกเปลี่ยนข้อมูลระหว่างกัน ซึ่งแต่เดิมมีวิธีการแลกเปลี่ยนข้อมูลกันตามรูปแบบที่เรียกว่า อีดีไอ (EDI) แต่จากการที่มีการใช้เอ็กซ์เอ็มแอล ทำให้การทำงานได้ตรงตามความต้องการ

ลักษณะสำคัญของการบริการบนเว็บจึงเป็นงานประยุกต์ที่เซิร์ฟเวอร์บริการสามารถเชื่อมโยงไปกับการประยุกต์อื่น ๆ โดยสามารถเรียกหาข้อมูลจากที่อื่นเพื่อนำมาประมวลผล และส่งต่อให้ผู้ใช้ที่เรียกเข้ามาต่อไป

2.1.2 เว็บเซอร์วิส

การพัฒนาทางเทคโนโลยีใดๆ ล้วนแล้วแต่มีการพัฒนาเป็นยุคสมัย แต่ละยุคสมัยก็จะมีการใช้เทคโนโลยีที่มีจุดสำคัญของการพัฒนาที่เด่นชัด การพัฒนาเทคโนโลยีประยุกต์ใช้งานบนเว็บก็เช่นเดียวกัน เริ่มจากยุคแรกที่เป็นแบบสแตติกเว็บเพจที่เป็นเพียงแค่ไฟล์เอชทีเอ็มแอลไว้ให้ผู้ดูข้อมูลเฉยๆ จนสู่ยุคที่ 2 เป็นแบบไดนามิกเว็บเพจ (Dynamic Web Page) ที่สามารถแปรเปลี่ยนตามคำร้องขอของผู้ใช้งานได้ ซึ่งเป็นการเพิ่มประสิทธิภาพของเว็บเพจ จนกระทั่งในปัจจุบันมีการพัฒนาเว็บเพจเข้ามาสู่ยุคที่ 3 หรือแบบการบริการผ่านเว็บเซอร์วิส โดยจุดประสงค์เพื่อรองรับความต้องการการใช้งานที่สูงขึ้น และเป็นมาตรฐานในการทำงานร่วมกัน

เว็บเซอร์วิส คือ ซอฟต์แวร์คอมโพเนนท์ (Software Component) ที่ผู้ให้บริการนำมาสร้างเป็นแอปพลิเคชันสำหรับให้บริการการทำงานหนึ่งๆ ให้กับผู้ใช้บริการทางอินเทอร์เน็ต หรือผู้ใช้บริการสามารถที่จะขอบริการจากหลายๆ ที่มาประกอบกันได้ โดยที่แต่ละระบบนั้นมีความเป็นอิสระจากกัน (Loosely Coupled) ซึ่งปัจจัยพื้นฐานของเว็บเซอร์วิสที่ควรมี ซึ่งมีความเกี่ยวข้องกันในทางเทคนิคและทางธุรกิจคือ

- การรวมซอฟต์แวร์ต่างระบบกัน จะต้องอนุญาตให้แต่ละระบบมีความเป็นอิสระจากกัน
- อินเทอร์เน็ตทางด้านการบริการของซอฟต์แวร์ควรจะเผยแพร่สู่สาธารณชนและสามารถเข้าถึงได้ง่าย
- เมสเสจที่ใช้ติดต่อกันของการทำงานแบบแอปพลิเคชันกับแอปพลิเคชันต้องสอดคล้องกับมาตรฐานเปิดบนอินเทอร์เน็ต
- แอปพลิเคชันสามารถสร้างได้จากการใช้ซอฟต์แวร์คอมโพเนนท์ทั้งจากภายในและภายนอกองค์กร โดยสร้างตามการดำเนินธุรกิจหลักขององค์กร
- มีแหล่งซอฟต์แวร์คอมโพเนนท์ที่หาได้ง่าย ซึ่งช่วยเพิ่มความยืดหยุ่นในการสร้างแอปพลิเคชันที่มีคุณลักษณะเฉพาะตามกระบวนการดำเนินการทางธุรกิจ
- การนำซอฟต์แวร์จากภายนอกมาใช้ใหม่ช่วยให้เกิดการลดต้นทุนและช่วยปรับปรุงบริการให้แก่ลูกค้า
- ซอฟต์แวร์สามารถขายเป็นบริการได้

หลักการพื้นฐานของเทคโนโลยีต่างๆ ที่พัฒนามาสู่เว็บเซอร์วิส

- การพัฒนาโปรแกรมแบบซอฟต์แวร์คอมโพเนนท์ ตามแนวคิดของการออกแบบและพัฒนาโปรแกรมเชิงวัตถุ (Object-Oriented Concept)
- การออกแบบระบบแบบกระจายจากศูนย์กลาง (Distributed Computing) ซึ่งเป็นเป้าหมายสำคัญของการพัฒนาระบบตามสถาปัตยกรรมแบบไคลเอนต์เซิร์ฟเวอร์ (Client-Server)

- การทำอีดีไอ หรือ อิเล็กทรอนิกส์ดาต้าอินเทอร์เชนจ์ (Electronic Data Interchange) ซึ่งสร้างขึ้นโดยกำหนดรูปแบบและมาตรฐานของข้อมูลสำหรับการทำธุรกิจ
- การบูรณาการของซอฟต์แวร์ต่างระบบ หรือ เอนเทอร์ไพรส์ แอปพลิเคชัน อินทิเกรชัน : อีเอไอ (Enterprise Application Integration : EAI) ที่อยู่บนพื้นฐานของความต้องการใช้ข้อมูลร่วมกัน รวมทั้งการแลกเปลี่ยนข้อมูลระหว่างแอปพลิเคชันให้สามารถทำงานได้อย่างถูกต้องเหมาะสม
- รูปแบบการให้บริการซอฟต์แวร์แบบเอสพี (ASP) หรือ แอปพลิเคชันเซอร์วิสโพรไวเดอร์ (Application Service Provider)
- แนวคิดการทำเหมืองข้อมูล (Data Mining) ซึ่งต้องการนำข้อมูลที่ถูกจัดเก็บไว้ในรูปแบบที่แตกต่างกันตามแหล่งต่างๆมาใช้งานร่วมกัน

โดยที่เว็บเซอร์วิสได้นำหลักการต่างๆ เหล่านี้มาดำเนินการปรับปรุง แก้ไขข้อจำกัดต่างๆ โดยการคิดค้นเทคโนโลยีที่เป็นมาตรฐานเปิด (Opened Standard) และไม่มีใครเป็นเจ้าของ (Neural Vendor)

2.1.3 ลักษณะของเว็บเซอร์วิส

การพัฒนาเว็บเซอร์วิสใช้สถาปัตยกรรมบริการในลักษณะที่เรียกว่า สถาปัตยกรรมของแนวคิดทางด้านบริการ หรือ เอสโอเอ (Service-Oriented Architecture : SOA) เป็นแนวคิดเบื้องต้น แอปพลิเคชันส่วนใหญ่ในโลกของธุรกิจที่ใช้งานในปัจจุบัน เป็นแอปพลิเคชันและระบบย่อยที่ถูกสร้างขึ้น มีการทำงานที่ต้องสัมพันธ์กันอย่างไม่เป็นอิสระจากกัน การเปลี่ยนแปลงการทำงานในระบบย่อยหรือแอปพลิเคชันหนึ่ง อาจจะมีผลกระทบกับอีกแอปพลิเคชันหนึ่ง หรือบางครั้งอาจจะกระทบทั้งระบบ ทำให้การบำรุงรักษานั้นทำได้ยาก และมีต้นทุนที่สูงขึ้น รวมทั้งยังเป็นข้อจำกัดในการเชื่อมต่อกับระบบของคู่ค้าอื่นๆ

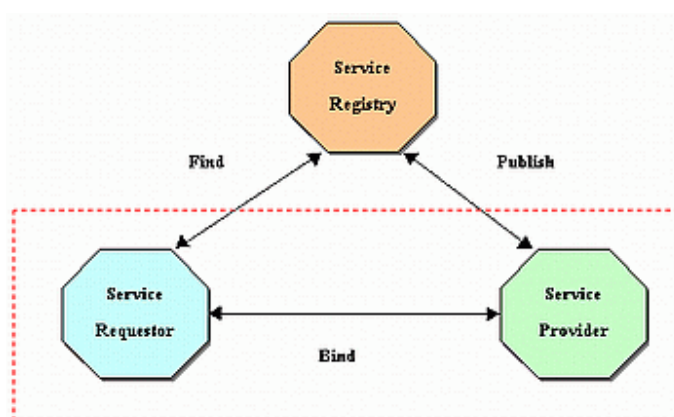
เอสโอเอไม่ใช่แนวคิดใหม่ แต่ได้เกิดขึ้นมานานแล้ว ซึ่งอยู่ในส่วนหนึ่งของแนวคิดการออกแบบระบบแบบกระจายจากศูนย์กลาง (Distributed Computing Concepts) เอสโอเอนั้น เป็นแนวคิดที่ได้รับการยอมรับและประสบความสำเร็จในระดับหนึ่ง การพัฒนาเอสโอเอ นั้นทำกันบนมาตรฐานเปิด ซึ่งได้รับการรับรองจากผู้ผลิตซอฟต์แวร์ชั้นนำและยังได้ร่วมมือกันรับรองมาตรฐานของเทคโนโลยีต่างๆ ที่ประกอบกันเป็นเว็บเซอร์วิส เช่น ซิมเปล็อบเจกต์เอกเชสโพรโตคอล (SOAP), ดับเบิลยูเอสดีแอล (WSDL) และ ยูดีดีไอ (UDDI) เป็นต้น

เอสโอเอมีส่วนประกอบหลัก 3 ส่วนคือ

1. ผู้ให้บริการ หรือที่เรียกว่า เซอร์วิสโพรไวเดอร์ (Service Provider)
2. ผู้ขอใช้บริการ หรือที่เรียกว่า เซอร์วิสรีเควสเตอร์ (Service Requester)
3. ตัวแทนของผู้ให้บริการ หรือที่เรียกว่า เซอร์วิสโบรกเกอร์ (Service Broker)

โดยส่วนประกอบหลักทั้ง 3 ส่วนนี้ สามารถติดต่อถึงกันได้โดยใช้ฟังก์ชันพื้นฐาน (Primary Function) คือการประกาศ (Publish) การค้นหา (Find) และการเรียกใช้ (Bind) ซึ่งฟังก์ชันทั้ง 3 มีการทำงานดังนี้

1. ผู้ให้บริการทำการประกาศบริการที่ตนเองให้บริการ ไปยังตัวแทนของผู้ให้บริการ ซึ่งตัวแทนของผู้ให้บริการจะทำการบันทึกเก็บไว้ในไดเรกทอรีของบริการ (Directory Service)
2. ผู้ขอใช้บริการจะทำการค้นหาบริการจากตัวแทนของผู้ให้บริการ
3. เมื่อพบบริการที่ต้องการแล้ว ผู้ให้บริการและผู้ขอใช้บริการจะทำการติดต่อกัน โดยผู้ขอใช้บริการจะทำการเรียกใช้บริการไปยังผู้ให้บริการนั้น



รูปที่ 2-2 แสดงความสัมพันธ์ของผู้ให้บริการ ผู้ขอใช้บริการ และตัวแทนผู้ให้บริการ

2.1.4 เทคโนโลยีพื้นฐานของเว็บเซอร์วิส

จากแนวคิดของเอสโอเอ ถูกนำมาใช้เป็นหลักการพื้นฐานของการพัฒนาเทคโนโลยีด้านเว็บเซอร์วิส ซึ่งเทคโนโลยีที่ถูกพัฒนาขึ้นมานี้เป็นมาตรฐานเปิดบนอินเทอร์เน็ตที่เกิดจากการทำงานร่วมกันของนักวิจัย และที่ปรึกษาจากบริษัทซอฟต์แวร์ต่างๆ

- เอกซ์เอ็มแอล หรือ เอกซเทนซิเบิลมาร์กอัพแลงเกจ เป็นภาษามาร์กอัพ ที่เป็น เท็กซ์เบส (Text-Based) ที่ใช้เป็นมาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตในปัจจุบัน ผู้ที่ทำหน้าที่รับผิดชอบ และกำหนดมาตรฐานของ เอกซ์เอ็มแอล คือ ดับเบิลยูทีซี หรือ เวิลด์ไวด์เว็บคอนซอร์ติอัม (W3C - World Wide Web Consortium)
- โซป หรือ ซิมเพลออปเจกต์แอคเซสโปรโตคอล เป็นโปรโตคอลที่สร้างบนพื้นฐานของเอกซ์เอ็มแอล และยังเป็นแมสเซจิงโปรโตคอล (Messaing Protocol) สำหรับใช้ในการแลกเปลี่ยนข้อมูลในสภาวะแวดล้อมแบบกระจายศูนย์ (Decentralized, Distributed Environment) โดยโซป ได้กำหนดแมสเซจิงโปรโตคอลระหว่างผู้ขอบริการกับผู้ให้บริการในการติดต่อสื่อสารกัน เช่น

กำหนดให้ผู้ขอใช้บริการต้องส่งข้อมูล เช่น ข้อมูลที่ระบุฟังก์ชัน และค่าพารามิเตอร์ต่างๆที่ต้องใช้ในแอปพลิเคชันที่ร้องขอ ส่งไปให้กับผู้ให้บริการ

- **ดับเบิลยูเอสดีแอล หรือ เว็บเซอร์วิสเดสคริปต์ขั้นสูง (WSDL - Web Services Description Language)** เป็นภาษาที่ใช้อธิบายคุณลักษณะการให้บริการของเว็บเซอร์วิส และวิธีการติดต่อขอรับบริการจากเว็บเซอร์วิส ดับเบิลยูเอสดีแอลสร้างขึ้นโดยใช้ภาษาเอ็กซ์เอ็มแอล ซึ่งเกิดจากการรวมแนวความคิดของเอนเอเอสเอสแอล หรือ เน็ตเวิร์คแอคเซสซิเบิลเซอร์วิสสเปคซิฟิเคชันแลงกวัย (NASSL - The Network Accessible Service Specification Language) และ ดับเบิลยูเอสดี หรือ เวลดีไฟด์เซอร์วิส (WDS - Well-Defined Services) ของบริษัทไอบีเอ็มกับเอสดีแอล หรือ เซอร์วิสเดสคริปต์ขั้นสูง (SDL - The Service Description Language) และ เอสซีแอล หรือ โซปคอนแทร็กต์แลงกวัย (SCL - The SOAP Contract Language) ของบริษัทไมโครซอฟต์ ปัจจุบันดับเบิลยูเอสดีแอลเป็นภาษาที่อยู่ในการดูแลขององค์กรดับเบิลยูทีซี

2.2 เอ็กซ์เอ็มแอล (XML)

มาร์กอัปแลงกวัย (Markup language) คือ ภาษาที่ใช้อธิบายความหมายของเอกสารหรือข้อมูล เราจะพบอยู่ในรูปของแท็ก (tag) เป็นส่วนมาก มาร์กอัปแลงกวัยมีข้อดีคือสามารถใช้ได้กับทุกแพลตฟอร์ม วัฒนาการของ มาร์กอัปแลงกวัยนั้น เริ่มต้นจากจีเอ็มแอล (GML) หรือ เจเนอไรซ์มาร์กอัปแลงกวัย (Generalized Markup Language) ซึ่งมีแนวคิดคือผู้ใช้สามารถคิดกลุ่มของแท็กขึ้นมาเองเพื่อใช้อธิบายเอกสาร ต่อมามีการพัฒนาจีเอ็มแอล ให้มีความเป็นมาตรฐานขึ้นจนกลายเป็น เอสจีเอ็มแอล (SGML) หรือ สแตนดาร์ดเจเนอไรซ์มาร์กอัปแลงกวัย (Standard Generalized Markup Language) ซึ่งมี ดีทีดี (DTD) หรือ ค็อกคิวเมนต์ไทป์เดฟิเนชัน (Document Type Definition) เป็นตัวนิยามโครงสร้างของเอกสาร หรือสามารถพูดได้อีกอย่างว่า เราจะอาศัย ดีทีดี เป็นต้นแบบในการสร้างเอกสารเอสจีเอ็มแอล

ต่อมา ในปี ค.ศ .1991 ทิม เบอร์เนอลี (Tim Berners-Lee) แห่งสถาบันวิจัยเซิร์น (CERN) ประเทศ สวิตเซอร์แลนด์ได้นำพื้นฐานของภาษาเอสจีเอ็มแอล ผสมเข้ากับความสามารถของไฮเปอร์เท็กซ์ (Hypertext) และใช้ยูอาร์แอล (URL) เพื่อการเชื่อมโยงระหว่างคอมพิวเตอร์หลายๆเครื่อง จนกลายมาเป็นเอชทีเอ็มแอล หรือ ไฮเปอร์เท็กซ์มาร์กอัปแลงกวัย (Hypertext Markup Language) โดยมีการสร้างกลุ่มของแท็ก หรือ เอลเมนต์ (Element) ขึ้นเพื่อใช้ในการแสดงผลบนเว็บผ่านเว็บเบราว์เซอร์ (Web Browser) ซึ่งนับว่าเป็นมาตรฐานที่มีประสิทธิภาพในการใช้ออกแบบเว็บเพจ

2.2.1 ข้อจำกัดของเอชทีเอ็มแอล

ถึงแม้ว่าเราจะสามารถสร้างเว็บเพจ ได้อย่างสวยงามและหลากหลายด้วยเอชทีเอ็มแอล แต่ความสามารถที่แท้จริงของเอชทีเอ็มแอล ก็เพียงแค่การแสดงผลทางเว็บเบราว์เซอร์เท่านั้น ข้อมูลที่นิยามด้วย

แท็กเอชทีเอ็มแอล ไม่สามารถจะไปใช้งานอย่างอื่นได้เลย นอกจากนี้เอชทีเอ็มแอลก็ยังมีข้อจำกัดหลายประการซึ่งทำให้มันยังไม่สามารถตอบสนองความต้องการของผู้ใช้ได้เต็มที่ ดังนี้

- เราไม่สามารถกำหนดแท็กขึ้นมาเองได้ ทำให้การทำงานมีขีดจำกัดแท็กของเอชทีเอ็มแอล มีความหมายที่กำหนดไว้แล้วชัดเจนโดยองค์กรดับบิวทรีซี หรือ เวิลด์ไวด์เว็บคอนซอร์ตียม และเป็นมาตรฐานเดียวกันเพื่อให้เว็บเบราว์เซอร์ สามารถแปล ความหมายของแท็กที่อยู่ในเว็บเพจและแสดงผลได้อย่างถูกต้อง
- เอชทีเอ็มแอลถูกออกแบบมาเพื่อจัดรูปแบบข้อมูลในการแสดงผลเป็นหลัก เช่น บอกให้เบราว์เซอร์รู้ว่าต้องจัดข้อความใดเป็นตัวหนาตัวเอียงเท่านั้นไม่ได้สื่อความหมายของข้อมูลเอกสาร ถ้าไม่มีเว็บเบราว์เซอร์ ภาษาเอชทีเอ็มแอลก็ไม่มี ความหมายเลย
- เอชทีเอ็มแอลไม่จำเป็นต้องอยู่ในรูปแบบที่ฟอร์มที่ถูกต้อง (Well-Formed) ก็ยังสามารถทำงานได้ ทำให้มีปัญหาคือ เมื่อแสดงผลไฟล์เอชทีเอ็มแอลเดียวกันจากเบราว์เซอร์ต่างกัน ผลลัพธ์ออกมาไม่เหมือนกัน เพราะมีความตีความแตกต่างกันไป
- เอชทีเอ็มแอลไม่เหมาะกับการที่ต้องมีการเปลี่ยนแปลงข้อมูลบ่อย เพราะการแก้ไขเอกสารแต่ละครั้งทำได้ยาก
- เมื่อแปลงข้อมูลจากไฟล์ข้อความธรรมดาเป็น เอชทีเอ็มแอล แล้ว ในการแปลงจากไฟล์ เอชทีเอ็มแอล กลับมาเป็นไฟล์ข้อความธรรมดาทำได้ยาก

จากข้อจำกัดเหล่านี้ทำให้องค์กรดับ บิวทรีซี มีความเห็นว่าแนวโน้มต่อไปของภาษาคควรจะยืดหยุ่นได้มากกว่าภาษาเอชทีเอ็มแอล จึงได้สร้างภาษาเอ็กซ์เอ็มแอล หรือ เอ็กซ์เทนซิเบิลมาร์กอัปแลงเกจขึ้นมา โดยมีพื้นฐานมาจากภาษา 2 ภาษา คือ ภาษาเอสจีเอ็มแอลโดยเฉพาะในแง่ของสามารถในการนิยามของข้อมูล และ ภาษาเอชทีเอ็มแอล

2.2.2 ลักษณะที่สำคัญของเอ็กซ์เอ็มแอล

เอ็กซ์เอ็มแอล เป็นภาษามาร์กอัปที่เป็นเท็กซ์เบสที่ใช้เป็นมาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตในปัจจุบัน โดยมีผู้ที่ทำหน้าที่รับผิดชอบและกำหนดมาตรฐานของเอ็กซ์เอ็มแอล คือ องค์กรดับบิวทรีซี

- จุดประสงค์ในการออกแบบ เพื่อกำหนดเป็นมาตรฐานในการทำเอกสารบนเครือข่ายอินเทอร์เน็ตเพื่อการแลกเปลี่ยนข้อมูลที่เป็นลำดับขั้น ดังนั้นจึงต้องมีรูปแบบที่สามารถใช้ได้ทั่วไปบนอินเทอร์เน็ต

- มีคุณลักษณะไม่ขึ้นกับแพลตฟอร์ม (Platform Independence) สามารถนำไปใช้กับคอมพิวเตอร์ระบบใด แพลตฟอร์มไหนก็ได้ เนื่องจากเอกสารเอ็กซ์เอ็มแอล เป็นเท็กซ์ไฟล์ (text file) ธรรมดา
- เป็นภาษาที่ได้รับการออกแบบมาเพื่อให้สามารถนิยามความหมายของข้อมูลได้ จึงมีการจัดโครงสร้างข้อมูล แบ่งข้อมูลออกเป็นหมวดหมู่และส่วนประกอบย่อย
- ไม่มีแท็กที่ถูกระบุไว้ก่อน อนุญาตให้ผู้สร้างแท็กขึ้นมาเองเพื่อใช้อธิบายข้อมูลได้ โดยที่ผู้กำหนดแท็ก และใช้งานได้ทันที เนื่องจากแท็กเป็นอะไรก็ได้ที่ผู้กำหนดจึงทำให้ เอ็กซ์เอ็มแอล ขยายขีดความสามารถต่อไปได้ (Extensible) และข้อมูลสามารถอธิบายความหมายข้อมูลของตัวเองได้ (Self Describe)
- ส่วนข้อมูลและส่วนการแสดงผลของเอกสารเอ็กซ์เอ็มแอล ถูกแยกออกจากกันอย่างชัดเจน ในเอกสารเอ็กซ์เอ็มแอล นั้น จะมีแต่ตัวเนื้อข้อมูล ส่วนการแสดงผลนั้น เราสามารถใช้สไตลชีท (style sheet) ได้หลายประเภท เช่น ซีเอสเอส (CSS), เอ็กซ์เอสแอล (XSL) เป็นต้น
- เนื่องจากเอกสารเอ็กซ์เอ็มแอล เป็นเพียงข้อความง่าย ๆ ที่ประกอบด้วยแท็กบางอย่างเท่านั้นจึงสามารถสร้างเอกสารเอ็กซ์เอ็มแอล ด้วยเท็กซ์อีดิเตอร์ (text editor) ทั่วๆไปได้
- การอ่านและแปลความหมายของเอกสาร เอ็กซ์เอ็มแอล สามารถทำได้โดยใช้โปรแกรมที่เรียกว่า เอ็กซ์เอ็มแอลพาร์เซอร์ (XML Parser) ได้ ตัวอย่างของโปรแกรมประเภท เอ็กซ์เอ็มแอลพาร์เซอร์ เช่น เอ็มเอสเอ็กซ์เอ็มแอล (MSXML) ซึ่งอยู่ในอินเทอร์เน็ตเอ็กซ์พลอเรอร์ (Internet Explorer) ของไมโครซอฟท์ และ เจเอเอ็กซ์พี (JAXP) ของบริษัทซัน (Sun) เป็นต้น
- มีวิธีกำหนดโครงสร้างเอกสาร 2 วิธีคือ ดีทีดี และ เอ็กซ์เอ็มแอลสคริม่า (XML Schema) ซึ่งไม่ได้บังคับว่าจำเป็นต้องมีไฟล์กำหนดโครงสร้างเอกสาร แต่ถ้ามีและเอกสารเอ็กซ์เอ็มแอล มีรูปแบบถูกต้องตาม ดีทีดี หรือ เอ็กซ์เอ็มแอลสคริม่า จะถือว่าเอกสารเอ็กซ์เอ็มแอลนั้นมีคุณสมบัติถูกต้อง (valid)
- มีความกะทัดรัด เข้าใจง่ายและใช้ประโยชน์ได้กว้างขวาง
- สามารถใช้ได้หลายภาษาผสมกัน เนื่องจากเอ็กซ์เอ็มแอล สนับสนุน ยูนิโคด(UNICODE)
- ได้รับการสนับสนุนในโปรแกรมระบบฐานข้อมูลหลายๆค่าย สามารถดึงข้อมูลจากฐานข้อมูล (Database) มาอยู่ในรูปของเอ็กซ์เอ็มแอลได้

2.2.3 ประโยชน์ของเอ็กซ์เอ็มแอล

ประโยชน์สำคัญของ เอ็กซ์เอ็มแอล ความจริงแล้วมีเพียงอย่างเดียวคือ การเป็นแม่แบบหรือต้นแบบในการนิยามข้อมูลเพื่อใช้งานต่างๆ ซึ่งเป็นประโยชน์ที่ครอบคลุมที่สุด แต่สามารถแจกแจงได้หลายประการได้แก่

1. ใช้สำหรับสร้างข้อมูลที่สามารถอธิบายความหมายของตัวเองได้ (Self-Describe Data) จากการที่เราสามารถกำหนดแท็ก มาอธิบายข้อมูลที่อยู่ภายในแท็กเองได้ ทำให้ข้อมูลมีความหมายในตัวเอง และสามารถเขียนโปรแกรมมาดึงข้อมูลไปใช้งานง่าย และคนทั่วไปก็อ่านได้เข้าใจ

2. ใช้สำหรับการแลกเปลี่ยนข้อมูล (Data Exchange) ด้วยความที่เป็นไฟล์ข้อความธรรมดา ทำให้เอกสารเอ็กซ์เอ็มแอล เป็นภาษากลางที่สามารถใช้ได้ทุกแพลตฟอร์ม จึงไม่แปลกที่เราจะสามารถแลกเปลี่ยนข้อมูลข้ามแพลตฟอร์มได้

3. เป็นรูปแบบข้อความในการสื่อสาร (Messaging Format) ระหว่างแอปพลิเคชัน หรือ โปรแกรม เอ็กซ์เอ็มแอลเป็นรูปแบบการสื่อสารระหว่างองค์ประกอบต่างๆ ตามแนวคิดของเว็บเซอร์วิส นอกจากเอ็กซ์เอ็มแอลแล้ว ยังมีมาตรฐานต่างๆ ที่เป็นส่วนสำคัญในการทำงานของเว็บเซอร์วิส เช่น โซป หรือ ซิมเปลโอเปอติงแอสโซซิเอชันโปรโตคอล และ ยูดีดีไอ หรือ ยูนิเวอร์ซัลเดสคริปต์ซันดิสคัฟเวอรีแอนคอินทิเกรชัน ซึ่งทั้ง โซป และ ยูดีดีไอ ล้วนมีพื้นฐานมาจาก เอ็กซ์เอ็มแอล เช่นกัน

4. ประโยชน์ในเชิงเทคโนโลยีอินเทอร์เน็ตและการพัฒนาเว็บ

5. เป็นรากฐานของภาษาใหม่ๆ ในการพัฒนาเว็บ ภาษาใหม่ๆ นี้ได้แก่ ภาษา เอ็กซ์เอชทีเอ็มแอล (XHTML), แมทเอ็มแอล (MathML คือ กลุ่มของแท็กเพื่อใช้นิยามเครื่องหมายในทางคณิตศาสตร์ขั้นสูง), วีเอ็มแอล (VML คือภาษาที่ใช้วาดรูปกราฟิกเพื่อแสดงผ่านเว็บเบราว์เซอร์) และ ดับเบิลยูเอ็มแอล (WML คือภาษาที่ใช้ในการสร้าง แวพไซต์ (Wap Site)) เป็นต้น

6. ใช้ในแวดวงธุรกิจแบบบีทูบี (B2B = Business to Business) กรณีนี้จะต้องใช้ภาษาเฉพาะ อย่างเช่น ซีเอ็มเอ็กซ์แอล (cXML = Commerce XML), เอ็กซ์ซีบีแอล (xCBL = XML Common Business Language) เป็นต้น โดยมีแท็กที่ใช้สนับสนุนการจัดการเกี่ยวกับแคตตาล็อกสินค้า และธุรกรรมเกี่ยวกับอีคอมเมิร์ซ (E-Commerce)

นอกจากนี้ ต่างประเทศยังมีโครงการนำ เอ็กซ์เอ็มแอลมา ใช้แทนระบบ อีดีไอ หรือ อิเล็กทรอนิกส์ดาต้าอินเทอร์เช้นต์ ซึ่งเป็นการแลกเปลี่ยนเอกสารอิเล็กทรอนิกส์ระหว่างองค์กร

2.2.4 ความแตกต่างระหว่างเอชทีเอ็มแอล และ เอ็กซ์เอ็มแอล

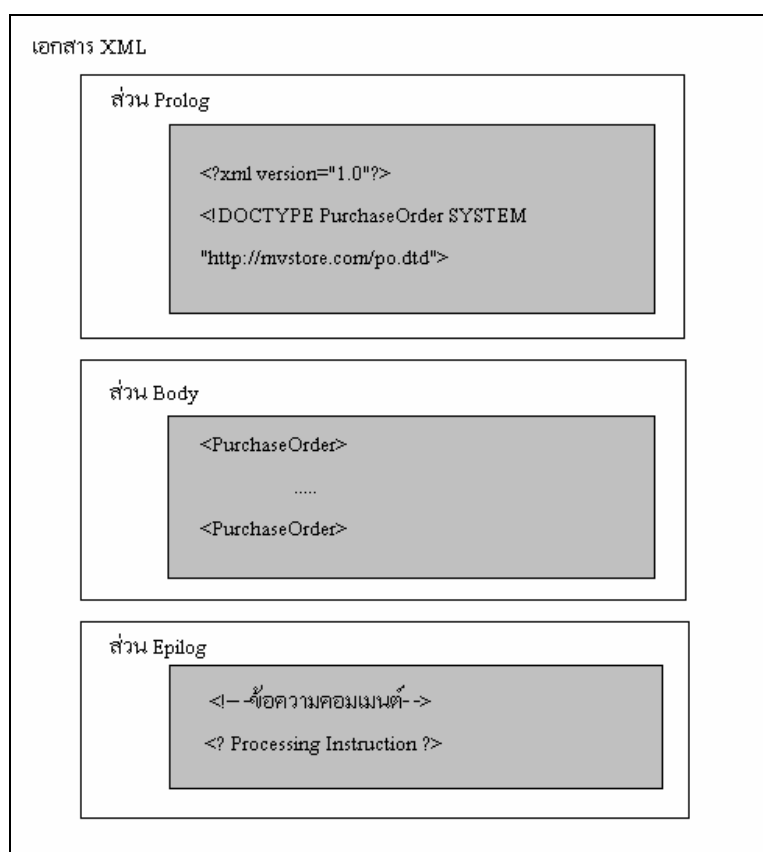
ถึงแม้ เอ็กซ์เอ็มแอล จะพัฒนามาจาก เอชทีเอ็มแอล แต่ทั้ง เอชทีเอ็มแอล และ เอ็กซ์เอ็มแอล ก็ล้วนมีวัตถุประสงค์หลักที่แตกต่างกัน เอชทีเอ็มแอล นั้นใช้เพื่อการสร้างเว็บ แต่ เอ็กซ์เอ็มแอล นั้นมีจุดประสงค์เพื่อใช้นิยามข้อมูล เอ็กซ์เอ็มแอล ไม่สามารถแสดงผลได้ด้วยตัวของมันเอง เพราะฉะนั้น เอ็กซ์เอ็มแอล ก็ไม่สามารถที่จะมาแทน เอชทีเอ็มแอลได้ และในบางกรณีทั้ง เอ็กซ์เอ็มแอล และ เอชทีเอ็มแอล ก็สามารถทำงานร่วมกันได้อย่างดี มีการพัฒนาภาษา เอ็กซ์เอชทีเอ็มแอล หรือ เอ็กซ์เทนซิเบิลไฮเปอร์เท็กซ์มาร์กอัปแลงเควด จากข้อดีของภาษา เอชทีเอ็มแอล และ เอ็กซ์เอ็มแอล รวมกันเพื่อใช้แสดงผลข้อมูลผ่านเว็บ แบบเดียวกับ

เอชทีเอ็มแอล โดยที่สามารถใช้แท็กของ เอชทีเอ็มแอล ได้ แต่ต้องทำตามกฎของ เอกซ์เอ็มแอล คือเขียนให้อยู่ในรูปแบบที่เป็นมาตรฐาน

2.2.5 โครงสร้างเอกซ์เอ็มแอล

โครงสร้างของเอกสารเอกซ์เอ็มแอล ประกอบด้วย 3 ส่วน

- โปรล็อก (Prolog) แบ่งเป็น ดีคลิเรชัน (Declaration) และ ค็อกคิวเมนต์ไทป์ดีคลิเรชัน (Document Type Declaration)
- บอดี้ (Body) เป็นส่วนของเนื้อเอกสารจริงๆ ได้แก่ข้อความหรือข้อมูลในเอกสาร และ แท็ก ที่นิยามข้อความหรือข้อมูลเหล่านั้น
- อีพิล็อก (Epilog) คือส่วนที่เป็นข้อความจำพวกคอมเมนต์ (Comment) และ พีไอ (PI) หรือ โปรเซสซิงอินสตรักชัน (Processing Instruction) จริงๆแล้วส่วนนี้ไม่จำเป็นต้องอยู่บริเวณล่างสุดของเอกสารดังในภาพ แต่คอมเมนต์ และ พีไอ จะแทรกอยู่ในส่วนบอดี้ของเอกสาร แต่ในภาพนี้แยกไว้ด้านล่างเพื่อให้เห็นว่าในเอกสาร เอกซ์เอ็มแอล หนึ่งๆ มีองค์ประกอบ 3 ส่วน



รูปที่ 2- 3 ส่วนประกอบของเอกสารเอกซ์เอ็มแอล

2.2.6 ข้อควรระวังในการเขียนเอ็กซ์เอ็มแอล

สิ่งที่สำคัญในการเขียน เอ็กซ์เอ็มแอล คือ คุณสมบัติเวลฟอร์มเนส (well-formedness) ซึ่งเกี่ยวข้องกับ การใช้แท็ก และเอลเลอเมนต์ต่างๆอย่างถูกต้อง ไม่ว่าจะเป็นการระบุแท็ก เปิด-ปิด การระบุค่าแอททริบิวต์ (Attribute) การตั้งชื่อแท็ก เป็นต้น การที่ เอ็กซ์เอ็มแอล มีกฎเคร่งครัดเช่นนี้ก็เพื่อให้เอกสารเป็นระเบียบแบบแผน สามารถสร้างตัวแปรภาษา หรือ เอ็กซ์เอ็มแอลพาร์เซอร์ ได้ง่าย ไม่ต้องมากังวลว่าจะจัดการกับเอกสารที่ ระบุแท็ก ไม่ถูกต้องอย่างไร เพราะฉะนั้น เอ็กซ์เอ็มแอลพาร์เซอร์ ทุกโปรแกรม จะทำงานกับเอกสาร เอ็กซ์เอ็มแอลที่เวลฟอร์มเท่านั้น ถ้าเอกสารไม่อยู่ในรูปเวลฟอร์ม จะเกิดข้อผิดพลาด (Error) ซึ่งการจะเขียนเอกสาร เอ็กซ์เอ็มแอล ให้มีคุณสมบัติเวลฟอร์มเนส มีข้อควรระวังต่อไปนี้

- เอกสารเอ็กซ์เอ็มแอล หนึ่งๆ จะมีรูทเอลเลอเมนต์ (root element) ได้เพียงอันเดียวเท่านั้น ซึ่งทำหน้าที่คลุมเอลเลอเมนต์อื่นๆทั้งหมด
- แท็กเปิด และ แท็กปิดต้องเหมือนกัน ต่างกันแต่เพียงในแท็กปิดต้องมีเครื่องหมาย (/) นำหน้าชื่อแท็กเท่านั้น
- ห้ามระบุแท็ก เหลื่อมซ้อนกัน (Overlap) คือ แท็กที่เปิดก่อน ต้องปิดทีหลัง
- ชื่อแท็กมีคุณสมบัติ เคสเซ็นซิทีฟ (case-sensitive) คือ ตัวอักษรพิมพ์เล็ก และพิมพ์ใหญ่ถือว่าแตกต่างกัน ดังนั้นแท็ก <Name>, <NAME>, <name> ถือว่าแตกต่างกัน
- สำหรับแท็กว่าง (Empty Tag) คือ แท็กที่ไม่มีข้อมูลข้างใน มีวิธีเขียนได้ 2 แบบ แบบหนึ่งคือ <title/> และ อีกแบบคือ <title></title> ส่วนมากจะนิยมแบบแรกมากกว่า
- ค่าของแอททริบิวต์ ต้องอยู่ในเครื่องหมายคำพูดแบบ (") หรือ (') อย่างใดอย่างหนึ่งเช่น <book isbn="123-45678-9-0"/> กรณีที่ในแอททริบิวต์ มีเครื่องหมายคำพูดแบบใดแบบหนึ่งอยู่ เราต้องเลี่ยงไปใช้เครื่องหมายคำพูดอีกแบบในการครอบค่าแอททริบิวต์
- ในภาษาเอ็กซ์เอ็มแอล มีอักขระซึ่งสงวนไว้ 5 ตัว เพราะต้องใช้เป็นส่วนประกอบตามโครงสร้างของภาษา ดังนั้นหากเนื้อความเอกสารจำเป็นต้องมีอักขระเหล่านี้ ก็ต้องพิมพ์เป็นชุดอักขระพิเศษแทน ซึ่งอักขระชุดพิเศษนี้เรียกว่า เอนทิตีรีเฟอเรนส์ (Entity Reference) ซึ่งอักขระ และ เอนทิตีรีเฟอเรนส์ ทั้ง 5 มีดังนี้

อักขระพิเศษ	Entity Reference
<	<
&	&
>	>
“	"
’	'

รูปที่ 2- 4 อักขระชุดพิเศษ และ เอนทิตีรีเฟอเรนส์

การตั้งชื่อแท็ก มีหลักเกณฑ์ดังนี้

- ชื่อแท็ก ต้องขึ้นต้นด้วยตัวอักษร หรือเครื่องหมายอันเดอร์สกออร์ underscore(_) เท่านั้น
- ตัวถัดไปต้องเป็นตัวอักษร ตัวเลข เครื่องหมายจุด(.) , เครื่องหมายขีดทึบ(-), เครื่องหมายอันเดอร์สกออร์ underscore(_), หรือเครื่องหมายโคลอน(:) เท่านั้น
- อักษร 3 ตัวแรกห้ามเป็นคำว่าเอ็กซ์เอ็มแอล ไม่ว่าจะเป็นตัวเล็กหรือใหญ่ เพื่อสับสนเอาไว้ใช้ในอนาคต
- การกำหนดแอททริบิวของเอ็กซ์เอ็มแอลดีคลาร์เรชั่น จะต้องเรียงตามลำดับที่กำหนดไว้เท่านั้น คือ เวอร์ชัน (Version) ก่อน แล้วจึงตามด้วยเอนโค้ดดิ้ง (Encoding) แล้วสุดท้ายจึงเป็นแอททริบิวสแตนดอลน (Attribute Standalone)

2.2.7 การกำหนดโครงสร้างของเอกสารเอ็กซ์เอ็มแอล

เนื่องจากการที่เอ็กซ์เอ็มแอลยอมให้ผู้ใช้งานสร้างแท็กขึ้นมาเอง ผู้ที่เกี่ยวข้องกับการใช้งานเอกสารเอ็กซ์เอ็มแอลทุกฝ่ายจึงต้องมีการตกลงกันก่อนว่าจะเก็บข้อมูลส่วนไหนไว้ในแท็กอะไร เมื่อได้ข้อสรุปแล้วจึงยึดถือปฏิบัติในการกำหนดโครงสร้างเอ็กซ์เอ็มแอล (หรือที่เรียกว่าสคริปต์) โดยมีวิธีกำหนดโครงสร้างเอกสาร 2 วิธี ได้แก่ ดีทีดี และ เอ็กซ์เอ็มแอลสคริปต์ เอกสารที่มีรูปแบบตามโครงสร้างที่กำหนดไว้ ไม่ว่าจะเป็นวิธีใดก็ได้แล้วแต่ เรียกว่าเอกสาร เอ็กซ์เอ็มแอล นี้มีคุณสมบัติแวดล้อม

ความจริงแล้วเอกสารเอ็กซ์เอ็มแอล ไม่ต้องมี ดีทีดี หรือ เอ็กซ์เอ็มแอลสคริปต์ ก็สามารถใช้งานได้เช่นกัน แต่เอกสารเอ็กซ์เอ็มแอล จะต้องมีความสมบูรณ์คุณสมบัติแวดล้อมอยู่แล้ว ส่วนคุณสมบัติแวดล้อมนั้น เป็นเพียงส่วนเสริมซึ่งถ้ามีก็จะดีมาก ถ้าหน่วยงานที่เกี่ยวข้องกับเอกสารนั้นไว้วางใจซึ่งกันและกัน ก็อาจไม่สนใจคุณสมบัติข้อนี้ ทำให้เอกสารเอ็กซ์เอ็มแอล ไม่จำเป็นต้องมี ดีทีดี หรือ เอ็กซ์เอ็มแอลสคริปต์ กำกับก็ได้ เราจะสรุปความสำคัญของ ดีทีดี และ เอ็กซ์เอ็มแอลสคริปต์ ได้ดังนี้

- เป็นการกำหนดรูปแบบโครงสร้างของเอกสาร เอกซ์เอ็มแอล หรือเรียกว่า คาด้าโมเดล (Data Model) ว่าประกอบด้วยแท็กอะไรบ้าง, มีแอททริบิวต์อะไรบ้าง, ค่าของแอททริบิวต์เป็นอะไรได้บ้าง, ลักษณะการซ้อนแท็กเป็นอย่างไรได้บ้าง เป็นต้น
- เป็นสัญญาาระหว่างองค์กร ที่จะแลกเปลี่ยนข้อมูลกันในรูปของเอกซ์เอ็มแอล เพื่อให้เอกสารที่สร้างขึ้นมามีรูปแบบตรงตามที่ตกลงกันไว้ และจะได้สามารถนำเอาเอกสารเอกซ์เอ็มแอลนั้นไปใช้งานอย่างถูกต้อง

2.2.8 ข้อดีและข้อเสียของดีทีดี และ เอกซ์เอ็มแอลสคริปต์

ดีทีดีนั้นเข้าใจได้ง่ายกว่าเอกซ์เอ็มแอลสคริปต์ จากรูปไฟล์ Sl.dtd ซึ่งเป็นไฟล์ดีทีดี ที่เอาไว้จัดรูปแบบโครงสร้างไฟล์ test.xml บรรทัดแรกประกาศว่า รุทเอลเลอเมนต์ที่มีโนดลูก (node) เป็น สติวเด็น (student) ซึ่งเอลเลอเมนต์สติวเด็น (element student) ก็มี แอททริบิวต์สติวเด็น (attribute studentid) ซึ่งมีชนิดเป็น ไอดี (ID) และ #REQUIRED แปลว่า ทุกๆ เอลเลอเมนต์สติวเด็น จะต้องมี แอททริบิวต์สติวเด็น บรรทัดที่ 3 เป็นการบอกว่า แอททริบิวต์สติวเด็น มีเอลเลอเมนต์ย่อยลงไปอีกเป็น พรีฟิก (prefix) , เอฟเนม (fname) และแอลเนม (lname) ส่วนบรรทัดที่เหลือเป็นการบอกชนิดข้อมูลของเอลเลอเมนต์ย่อยเหล่านี้ว่าเป็นข้อมูลชนิดพีซีดาต้า (PCDATA) ย่อมาจาก พาสคาร์เรคเทอร์ดาต้า (Parsed Character Data) หมายความว่าเนื้อหาส่วนที่เป็นข้อความนี้เอาไว้ให้ โปรแกรมเอกซ์เอ็มแอลพาร์เซอร์ อ่านเพื่อประมวลผล ดีทีดี มีข้อจำกัดหลายอย่าง เช่น รับรองชนิดของข้อมูล (Data Type) ได้น้อยกว่า, ไม่ได้ใช้ไวยากรณ์ของเอกซ์เอ็มแอล เมื่อเราต้องการแปลความหมายของดีทีดี เราจึงไม่สามารถแปลพร้อมกับเอกสารเอกซ์เอ็มแอล โดยใช้เอกซ์เอ็มแอลพาร์เซอร์ได้

```
<!ELEMENT StudentList (student)>
<!ATTLIST student studentid ID #REQUIRED>
<!ELEMENT student (prefix,fname,lname)>
<!ELEMENT prefix (#PCDATA)>
<!ELEMENT fname (#PCDATA)>
<!ELEMENT lname (#PCDATA)>
```

รูปที่ 2-5 ตัวอย่าง Sl.dtd

ส่วนเอกซ์เอ็มแอลสคริปต์จะรองรับชนิดของข้อมูลได้มากกว่าและหลากหลายกว่า, สามารถกำหนดรูปแบบข้อมูลเองได้, สร้างชนิดข้อมูลใหม่ได้, กำหนดจำนวนและลำดับของ เอลเลอเมนต์ลูก ได้ รวมทั้งใช้ไวยากรณ์ของเอกซ์เอ็มแอล และที่สำคัญที่สุดคือเอกซ์เอ็มแอลสคริปต์ สนับสนุน เนมสเปซ (namespace) เพื่อใช้สำหรับป้องกันความสับสนในการระบุชื่อองค์ประกอบใดๆ ภายในเอกสารเอกซ์เอ็มแอลคนละไฟล์ แต่ใช้

ชื่อเดียวกัน เพราะเป็นไปได้ว่าเอกสารเอ็กซ์เอ็มแอล ที่ใช้ในงานที่ต่างกัน อาจมีชื่อเอลเมนต์เหมือนกัน ชื่อชนิดข้อมูลเหมือนกัน แล้วอาจถูกเรียกใช้ภายในเอกสารเอ็กซ์เอ็มแอลไฟล์เดียวกัน แต่ เอ็กซ์เอ็มแอลสคริปต์ ก็มีข้อเสียคือเรียนรู้ได้ยากกว่าที่ดี

2.2.9 การแสดงผลเอกสารเอ็กซ์เอ็มแอลด้วยเอ็กซ์เอสแอล (XSL)

เอ็กซ์เอสแอล หรือ เอ็กซ์เทนซิเบิลสไตล์ชีทแลงกเวจ (Extensible Stylesheet Language) เป็นภาษาที่ใช้สำหรับกำหนดรูปแบบการแสดงผลเอกสารเอ็กซ์เอ็มแอล ไปเป็นเอกสารรูปแบบอื่นๆ เช่น เอกซ์เอ็มแอล ดับเบิลเอ็มแอล หรือแม้แต่การเปลี่ยนจากเอ็กซ์เอ็มแอล รูปแบบหนึ่งไปเป็นเอ็กซ์เอ็มแอล รูปแบบอื่นๆ เอ็กซ์เอสแอลมีความสำคัญเพราะโดยปกติแล้ว เอกสารเอ็กซ์เอ็มแอล ก็คือไฟล์ธรรมดาที่มีแท็กครอบข้อความอยู่เท่านั้น เราจึงต้องอาศัยเอ็กซ์เอสแอลมาช่วยในการนำเอกสารเอ็กซ์เอ็มแอล มาแสดงผลไม่ว่าจะผ่านทางบราวเซอร์หรืออะไรก็ตาม

ในการแสดงผลเอกสารเอ็กซ์เอ็มแอล นั้นมีทางเลือกสองทาง ทางแรกคือแสดงข้อมูลเอ็กซ์เอ็มแอล ทั้งส่วนแท็กและส่วนข้อมูลด้วย อินเทอร์เน็ตเอ็กซ์พลอเรอร์ 5 (Internet Explorer 5) ซึ่ง ไออี 5 (IE5) จะสร้างเอกสารแบบอินเทอร์แอคทีฟ (Interactive) ที่อนุญาตให้บราวเซอร์ทำงานกับข้อมูล, ยูน และ ขยายรายการภายในแท็กได้

ทางเลือกอีกทางคือ การใช้ เอ็กซ์เอสแอล เพื่อเปลี่ยนข้อมูล เอ็กซ์เอ็มแอล ให้เป็นเอกซ์เอ็มแอล เมื่อใช้ เอ็กซ์เอสแอล เราสามารถฝังข้อมูลเข้าไปในไฟล์ เอ็กซ์เอ็มแอล โดยตรง หรือใส่ข้อมูลเข้าไปในสไตล์ชีทเอ็กซ์เอ็มแอล (style sheet XML) แยกต่างหากโดย เอ็กซ์เอสแอล มีความสามารถเหมือนกับการใช้ ซีเอสเอส กับ เอกซ์เอ็มแอล แต่ยังช่วยให้เราสามารถแยกข้อมูลออกจากไฟล์เอ็กซ์เอ็มแอล เพื่อการแสดงผล นอกจากเอ็กซ์เอสแอล แล้ว เราก็สามารถใช้ ซีเอสเอส ได้ เช่นเดียวกัน ซีเอสเอส สามารถทำความเข้าใจได้ง่าย แต่มีข้อจำกัดในการจัดรูปแบบการแสดงผล ส่วนเอ็กซ์เอสแอล สามารถกำหนดลักษณะการแสดงผลได้หลากหลายกว่า เพราะเป็นการเปลี่ยนรูปแบบข้อมูล เช่น กรองข้อมูลได้ (Filter), เรียงลำดับข้อมูลได้ (Sort) และ จัดรูปแบบการแสดงผลได้ตามต้องการ

ทุกครั้งที่เปิดปิดดูเอกสารเอ็กซ์เอ็มแอลธรรมดา ด้วยไออี5 เราไม่จำเป็นต้องกำหนดสไตล์ชีทก็ได้ เพราะ ไออี5 มีสไตล์ชีทพิเศษฝังอยู่ข้างในอยู่แล้ว แต่ถ้าเรากำหนด สไตล์ชีท เข้าไปเอง ไม่ว่าจะเป็นแบบ ซีเอสเอส หรือ เอ็กซ์เอสแอล ก็ตามบราวเซอร์ ก็จะใช้ สไตล์ชีทของเราแทนของเราแทน ตามหลักการโอเวอร์ไรด์ (Override)

```
<?xml version=" "1.0encoding="windows-"874?>
<xsl:stylesheet xmlns:xsl="http://www.w.3org/TR/WD-xsl">
<xsl:template match="/">
```

```

        รายชื่อนักเรียน<hr/>

        <xsl:for-each select="StudentList/student">

            <p> <xsl:value-of select="fname"/>

                <xsl:value-of select="lname"/> </p>

        </xsl:for-each>

    </xsl:template>

</xsl:stylesheet>

```

รูปที่ 2-6 ตัวอย่างไฟล์ *slstyle.xml*

2.2.10 การแปลความหมายของ เอกซ์เอ็มแอลด้วยเอกซ์เอ็มแอลพาร์เซอร์

ในการพัฒนาแอปพลิเคชันโดยมีการนำเอกซ์เอ็มแอล ไปใช้งานนั้น สิ่งหนึ่งที่ผู้พัฒนาจำเป็นต้องทำคือการพาส (Parse) เอกสารเอกซ์เอ็มแอล เนื่องจากโปรแกรมมองเห็นเอกสารเอกซ์เอ็มแอล เป็นเพียงอักขระธรรมดาเท่านั้น เราจึงต้องมีการเขียนโปรแกรมเพื่อให้ แอปพลิเคชัน สามารถเข้าใจได้ว่าเอกสารเอกซ์เอ็มแอลของเรามีการจัดโครงสร้างเป็นอย่างไร โดยเราเรียกโปรแกรมนั้นว่าพาร์เซอร์

เอกซ์เอ็มแอลพาร์เซอร์ หรือ เอกซ์เอ็มแอลโพรเซสเซอร์ (XML Processor) เป็นตัวแปลเอกสาร เอกซ์เอ็มแอล ซึ่งทำหน้าที่พื้นฐานคือ อ่าน (Read) , สร้าง (Create) , แก้ไข (Update) และเข้าถึง (Manipulate) เอกสาร เอกซ์เอ็มแอล ปัจจุบันมี โปรแกรมเอกซ์เอ็มแอลพาร์เซอร์ หลาย โปรแกรม เช่น เอ็มเอสเอกซ์เอ็มแอลของไมโครซอฟต์ เป็นโปรแกรมที่พ่วงมากับ ไออี 5 ขึ้นไป สร้างโดยเทคโนโลยีคอม (COM = Component Object Model) , เอกซ์เอ็มแอลโฟร์เจ (XML4J = XML for Java) ของไอบีเอ็ม สร้างด้วยภาษาจาวา (Java Language) และ เจเอ็กซ์พี (JAXP = Java API for XML Parsing) ของบริษัทซัน (Sun) ซึ่งสร้างโดยภาษาจาวา เป็นต้น จากตัวอย่างที่ผ่านมา เราสามารถใช้โปรแกรม ไออี 5 ขึ้นไป กับเอกสารเอกซ์เอ็มแอล ของเราได้ก็เพราะว่าโปรแกรมนี้นี้มีการฝัง พาร์เซอร์ ชื่อ เอ็มเอสเอกซ์เอ็มแอล เอาไว้ในตัวอยู่แล้ว เราจึงไม่ต้องทำการกำหนด พาร์เซอร์ เหมือนเวลาเราเขียนโปรแกรมทั่วไป

การจำแนกชนิดของ เอกซ์เอ็มแอลพาร์เซอร์ แบ่งได้เป็นหลายวิธี แต่หลักเกณฑ์ที่นิยมกันมี 2 วิธี คือ

1. เอกซ์เอ็มแอลพาร์เซอร์ แบบ แวลิดเดตติ้ง (Validating) และ แบบนอนแวลิดเดตติ้ง (Non-Validating) ซึ่งหมายถึง พาร์เซอร์ ที่มีตรวจสอบคุณสมบัติเวลฟอร์มเนต และ คุณสมบัติแวลิดของเอกสาร ส่วน เอกซ์เอ็มแอลพาร์เซอร์ แบบนอนแวลิดเดตติ้ง หมายถึง เอกซ์เอ็มแอลพาร์เซอร์ ที่ไม่ตรวจสอบคุณสมบัติแวลิด และสนใจเฉพาะคุณสมบัติเวลฟอร์มเนต ถึงแม้ว่าเอกซ์เอ็มแอลพาร์เซอร์แบบแวลิดเดตติ้ง จะมีการตรวจสอบที่มากกว่า ก็ไม่ได้หมายความว่าดีกว่าเสมอไป ในบางกรณีก็ควรใช้ เอกซ์เอ็มแอลพาร์เซอร์ แบบนอนแวลิดเดตติ้ง เพราะว่า
 - ทำงานเร็วกว่า เพราะไม่ต้องมาคอยตรวจสอบโครงสร้างของเอกสาร

- ในกรณีที่มั่นใจอยู่แล้วว่าเอกสารนั้นอยู่ในรูปแบบที่ valide
 - ต้องการแก้ค้นหาที่บาง แท็กในเอกสารนั้นเท่านั้น ไม่ได้ต้องการตรวจสอบคุณสมบัติ valide โปรแกรมเอ็กซ์เอ็มแอลพาร์เซอร์ ส่วนใหญ่เป็นแบบ non-validating เพราะว่าสร้างได้ง่ายกว่า
2. เอ็กซ์เอ็มแอลพาร์เซอร์ แบบที่รองรับ คือคิกวเมนท้อบเจ็กต์โมเดล (DOM = Document Object Model) และแบบที่รองรับที่เรียกว่า แซก หรือ ซิมเปิลเอพีไอฟอร์เอ็กซ์เอ็มแอล (SAX – Simple API for XML) การแปลงเอกสารเอ็กซ์เอ็มแอล ต้องอาศัยโปรแกรม เอ็กซ์เอ็มแอลพาร์เซอร์ ซึ่งโปรแกรมจำพวกนี้มีวิธีการดึงข้อมูลจากเอกสารเอ็กซ์เอ็มแอล หลายวิธี แต่ที่นิยมกันมากมี 2 วิธี ก็คือ ค็อกวเมนท้อบเจ็กต์โมเดล และ ซิมเปิลเอพีไอฟอร์เอ็กซ์เอ็มแอล ซึ่งมีลักษณะต่างกันให้ เลือกใช้ตามความเหมาะสมของลักษณะงาน

2.2.11 การแปลงเอกสารโดยใช้ค็อกวเมนท้อบเจ็กต์โมเดล

ค็อกวเมนท้อบเจ็กต์โมเดล เป็น เอพีไอ (API = Application Programming Interface) สำหรับเอกสารเอ็กซ์เอ็มแอล ซึ่งจะกำหนดว่าเอกสารเอ็กซ์เอ็มแอล จะมีการเข้าถึง การโยกย้ายและการจัดการต่างๆ ได้อย่างไรบ้าง ด้วยวิธีการแบบค็อกวเมนท้อบเจ็กต์โมเดล ผู้พัฒนาจะมาสารถเข้าถึงโครงสร้างระดับต่างๆ ของเอกสารเอ็กซ์เอ็มแอลได้ นอกจากนั้นยังสามารถเพิ่ม ลบและแก้ไขเอลเลเมนต์ต่างๆ ของเอกสารได้อีกด้วย

ค็อกวเมนท้อบเจ็กต์โมเดล ได้รับการรับรองเป็นมาตรฐานโดย ดับเบิลยูทีซี มีหลักการคือเมื่อโปรแกรม หรือ แอปพลิเคชัน เรียกพาร์เซอร์ เพื่อทำการโหลด (Load) เอกสารเอ็กซ์เอ็มแอล เข้าสู่หน่วยความจำคอมพิวเตอร์ เมื่อเอกสารถูกโหลดเรียบร้อยแล้ว ค็อกวเมนท้อบเจ็กต์โมเดล ก็จะทำให้โปรแกรมสามารถดึงข้อมูลมาใช้งานได้ โดย ค็อกวเมนท้อบเจ็กต์โมเดล จะนำข้อมูลของเอกสารเอ็กซ์เอ็มแอลมาวางเป็นโครงสร้างทรีไดอะแกรม (Tree Diagram) ในหน่วยความจำของเครื่องคอมพิวเตอร์ ซึ่งทรีไดอะแกรม จะประกอบด้วยข้อมูลต่างๆ ในเอกสารเอ็กซ์เอ็มแอล เช่น เอลเลเมนต์ และ แอททริบิวต่างๆ เป็นต้น โดยแต่ละเอลเลเมนต์ก็จะเป็นโหนดในทรีไดอะแกรม ดังนั้นการเข้าถึงเอกสารเอ็กซ์เอ็มแอลจะรู้ว่าแต่ละเอลเลเมนต์มีค่าเป็นอะไร ก็คือการทราเวอร์ส (Traverse) ไปตามโหนดต่างๆ ในทรีไดอะแกรม จึงเรียกว่า เอ็กซ์เอ็มแอลพาร์เซอร์แบบที่รองรับ ค็อกวเมนท้อบเจ็กต์โมเดลเป็นทรีเบสพาร์เซอร์ (Tree-Base Parser)

ข้อดีของด็อกคิวเมนต์ออบเจกต์โมเดล

- หลังจากโปรแกรมทำการพาสข้อมูลแล้ว ก็จะเก็บทรีไคอะแกรมไว้ในหน่วยความจำตลอดจนกว่าเราจะสั่งยกเลิก เหมาะกับกรณีที่ต้องเข้าถึงข้อมูลหลายครั้ง เช่น ดึงข้อมูล แก้ไขข้อมูล หรือกรณีที่เราต้องการรู้ข้อมูลหลายอย่างเกี่ยวกับข้อมูลในเอกสารเอ็กซ์เอ็มแอล เพราะเมื่อเก็บเอกสารไว้ในหน่วยความจำแล้ว เราก็สามารถเข้าถึงข้อมูลในทรีไคได้อย่างรวดเร็ว
- กรณีที่ต้องมีการเคลื่อนย้ายข้อมูลบางส่วนของเอกสาร เช่น มีการเรียงข้อมูลใหม่ (Sort) เพราะการทำงานลักษณะนี้จะเกิดผลกระทบกับทั้งเอกสาร จึงควรที่จะโหลดเอกสารทั้งเอกสารเก็บไว้ในหน่วยความจำ
- ด็อกคิวเมนต์ออบเจกต์โมเดลสามารถสร้าง, เขียน และ อัปเดตข้อมูลเอ็กซ์เอ็มแอล จัดเก็บไว้ในไฟล์ และ สตรีม (Stream)
- เราสามารถเรียกใช้ข้อมูลแบบสุ่ม (Random) เนื่องจากข้อมูลทั้งหมดอยู่ในหน่วยความจำ
- ข้อมูลที่สร้างเป็นทรีไคทำให้ข้อมูลเกิดความสัมพันธ์ ทำให้การเรียกใช้ทำได้ง่าย

ข้อเสียของด็อกคิวเมนต์ออบเจกต์โมเดล

- เนื่องจากวิธีของด็อกคิวเมนต์ออบเจกต์โมเดลจะโหลดเอกสารเอ็กซ์เอ็มแอล ทั้งหมดมาเก็บในหน่วยความจำ กรณีที่เอกสารมีขนาดใหญ่ก็จะเปลืองเนื้อที่หน่วยความจำมาก การทำงานเป็นไปได้ช้า และในบางกรณีอาจไม่มีเนื้อที่เหลือพอจะทำงานได้

2.2.12 การแปลเอกสารโดยใช้ซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล

ซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอลไม่ได้เป็นมาตรฐานของหน่วยงานใด แต่เป็นวิธีที่นักพัฒนากลุ่มหนึ่งคิดค้นขึ้น และนำไปใช้กันมากจนได้รับการยอมรับทั่วไป ปัจจุบันเอ็กซ์เอ็มแอลพาร์เซอร์ จำนวนมากล้วนแต่รองรับซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล กันแล้ว

ซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอลจะไม่โหลดเอกสารเอ็กซ์เอ็มแอล เข้ามาในหน่วยความจำจึงไม่มีการสร้างภาพรวมของเอกสารไว้ก่อนว่าเอกสารนั้นมีโครงสร้างเป็นอย่างไร วิธีการของมันคือเมื่อแอปพลิเคชันมีการร้องขอมา มันก็จะอ่านเอกสารเอ็กซ์เอ็มแอลได้ตั้งแต่ต้นเอกสารและตัวพาร์เซอร์จะสร้างอีเวนต์ (Event) ให้กับจุดต่างๆ ที่สำคัญของเอกสารทุกๆ จุด เช่นทุกๆ เอลเลเมนต์เปิด เอลเลเมนต์ปิด ทุกๆ แอททริบิว เป็นต้น ลักษณะการทำงานเช่นนี้เรียกว่า อีเวนต์ไดรเวอร์พาร์เซอร์ (Event-Driven Parser)

ในทางปฏิบัติแล้วซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล มักไม่ได้มีการแปลเอกสารทั้งเอกสาร เมื่อเราได้อินโฟร์เมชันที่ต้องการแล้วเราก็จะหยุด วิธีการแปลแบบซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล จึงทำงานได้รวดเร็วและเขียนโปรแกรมได้ง่าย

ข้อดีของซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล

- ไม่เปลืองหน่วยความจำ เพราะไม่ได้โหลดทั้งเอกสารมาเก็บไว้เหมือน ค็อกคิวเมนต์ออบเจกต์โมเดลล์ จึงเหมาะกับกรณีที่เอกสารเอ็กซ์เอ็มแอล มีขนาดใหญ่ และเครื่องคอมพิวเตอร์มีหน่วยความจำไม่มาก
- ซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอลเข้าถึงข้อมูลได้ง่ายกว่า ค็อกคิวเมนต์ออบเจกต์โมเดลล์ เพราะการทำงานของ แชก จะมองเพียงจุดเริ่มต้นและจุดสิ้นสุดของเอกสารและของ เอเลเมนต์ และจะสนใจเฉพาะสิ่งที่ต้องการเท่านั้น จึงเหมาะกับกรณีต้องการใช้ข้อมูลจากเอเลเมนต์เพียงไม่กี่ตัวในเอกสาร ซึ่งถ้าใช้วิธี ค็อกคิวเมนต์ออบเจกต์โมเดลล์ จะเป็นงานที่กินแรงเกินความจำเป็น
- สามารถเขียนโปรแกรมได้ง่ายกว่า เพราะมองข้อมูลในลักษณะเส้นตรง

ข้อเสียของซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล

- ไม่สามารถบอกความสัมพันธ์ของข้อมูลได้
- เราไม่สามารถใช้ข้อมูลแบบสุ่มได้ เพราะไม่มีข้อมูลใดเหลืออยู่ในหน่วยความจำหลังจากที่
- ซิมเบิลเอพีไอฟอร์เอ็กซ์เอ็มแอล ได้ประมวลผลและสร้างอีเอนต์ที่มีความสัมพันธ์กันเสร็จแล้ว
- ยากที่จะสร้างฟังก์ชัน ในการค้นหาข้อมูล
- อ่านได้เพียงอย่างเดียว ไม่สามารถเขียนหรือ อัปเดตข้อมูลได้
- เป็นวิธีที่ไม่มีประสิทธิภาพถ้าต้องการเข้าถึงข้อมูลเดิมบ่อยครั้ง

2.3 ซิมเบิลออบเจกต์แอคเซสโปรโตคอล (Simple Object Access Protocol)

เนื่องจากจุดประสงค์หลักของการใช้งานเว็บเซอร์วิส เราต้องการให้แอปพลิเคชัน มีการทำงานกับแอปพลิเคชันที่ทำงานอยู่บนเครื่องอื่น โดยผ่านทางเครือข่าย ซึ่งเทคโนโลยีที่มีอยู่ปัจจุบันที่ใช้มีการสื่อสารระหว่างออบเจกต์ในระยะไกล หรือ อาร์พีซี (Remote Procedure Calls : RPC) เช่น ดีคอม (DCOM) , อีเจบี (EJB) หรือคอบร้า (COBRA) นั้นไม่ได้ถูกออกแบบมาใช้สำหรับโปรโตคอลเอชทีทีพี

เทคนิคอาร์พีซีของเทคโนโลยีที่กล่าวข้างต้นต่างก็มีปัญหาในด้านการนำมาใช้งานในแง่ของความเข้ากันได้ของการเรียกใช้งานข้ามเทคโนโลยี เนื่องจากเป็นเทคโนโลยีเฉพาะของแต่ละค่าย (ยกเว้น คอบร้า) ผู้พัฒนาระบบจะต้องพัฒนาโปรแกรมที่มีความซับซ้อน และยังมีปัญหาในส่วนของไฟร์วอลล์ และพร็อกซี่

เซิร์ฟเวอร์ ด้วยเนื่องจากโดยปกติเซิร์ฟเวอร์จะปิดการสื่อสารที่ไม่ใช่โปรโตคอลเอชทีทีพีออกไป เพื่อความปลอดภัยของระบบที่มีการติดต่อสื่อสารกับภายนอก

ดังนั้นทางเลือกของการสื่อสารที่จะนำมาใช้ในการให้บริการเว็บเซอร์วิส คือ ให้ทำงานอยู่บนโปรโตคอลเอชทีทีพี ซึ่งโซบ นอกจากจะทำงานบนโปรโตคอลเอชทีทีพีแล้ว ยังเป็นมาตรฐานเปิดที่จะทำให้สามารถติดต่อสื่อสารกับคอมพิวเตอร์ที่มีความแตกต่างทางระบบปฏิบัติการ, เทคโนโลยีรวมไปถึงภาษาที่ใช้ในการพัฒนา

โซบ หรือ ซิมเปิลอบเจกต์แอคเซส โปรโตคอล (SOAP = Simple Object Access Protocol) เป็นโปรโตคอลที่ใช้ เอ็กซ์เอ็มแอล เป็นพื้นฐาน เพื่อให้ซอฟต์แวร์คอมพิวเตอร์ และแอปพลิเคชันสามารถติดต่อกันผ่านเอชทีทีพี ซึ่งเป็นมาตรฐานอินเทอร์เน็ตโปรโตคอลได้ ซึ่งข้อดีของโซบก็คือโซบไม่ขึ้นกับคอมพิวเตอร์เทคโนโลยี และภาษาการเขียนโปรแกรมใดๆ สามารถเขียนได้ง่ายและขยายเพิ่มเติมได้

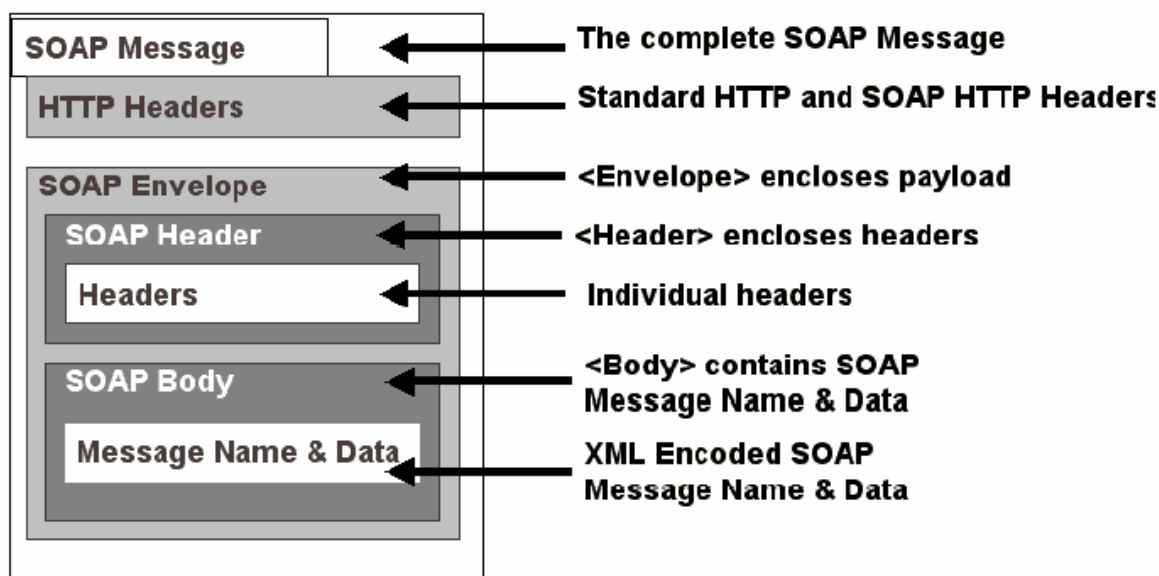
2.3.1 ส่วนประกอบของโซบ

โซบเมสเสจ ใช้ไวยากรณ์ของเอ็กซ์เอ็มแอล ในการสร้างประกอบด้วย 3 เอลเมนต์มาตรฐาน คือ โซบเอนวิล็อพ, โซบเฮดเคอร์, โซบบอดี

โซบนั้นประกอบด้วยกัน 3 ส่วนคือ

- โซบเอนวิล็อพ (SOAP Envelop) จะเป็นส่วนสำหรับใช้ในการระบุสิ่งที่อยู่ในเอกสาร ว่าจะต้องจัดการอย่างไร และบอกถึงความจำเป็นในการใช้งาน
- โซบเอนโค้ดดิ้งรูล (SOAP Encoding Rule) จะเป็นส่วนสำหรับกำหนดกลไกที่ใช้ในการแลกเปลี่ยนข้อมูล ว่ามีข้อตกลงอย่างไรในการแลกเปลี่ยนข้อมูล

โซบอาร์พีซีรีพรีเซนเทชัน (SOAP RPC Representation) จะเป็นส่วนสำหรับนิยามรูปแบบ รีโมท โปรซีเยอร์คอล (Remote Procedure Call) และ การตอบสนอง (Response)



รูปที่ 2-7 โครงสร้างของโซบ

โซบแมสเชจ จะต้องเป็นไปตามกฎนี้

- เอนวิล็พเป็นเอลเลอเมนต์ที่อยู่บนสุด ต้องมีเอลเลอเมนต์นี้เสมอ
- เซลเดออร์อาจจะมีหรือไม่มีก็ได้ แต่ถ้ามีต้องเป็นชายเอลเลอเมนต์ (Child Element) แรกของเอนวิล็พ
- บอดีต้องเป็นชายเอลเลอเมนต์แรกของเอนวิล็พ หรือ เซลเดออร์

ตัวอย่างของ โซบแมสเชจ ตัวอย่างนี้คือ GetLastTradePrice โซบรีเควส (SOAP Request) ที่ส่งไปยัง StockQuote เซอร์วิสประกอบด้วยสตริงพารามิเตอร์ ซิมโบ (Symbol) และ คีนค่าโฟลต (Float) กลับมากับ โซบเรสสปอน (SOAP Response) โซบเอนวิล็พเอลเลอเมนต์ที่อยู่จุดบนสุดของเอกสารเอ็กซ์เอ็มแอล เอ็กซ์เอ็มแอลเนมสเปซใช้เพื่อป้องกันการสับสนของโซบไอดีนิฟาย (SOAP Identifier) โซบรีเควสจะเป็นดังนี้

POST /StockQuote HTTP/1.1

HOST:www. Stockquoteserver.com

Content-Type: text/xml; charset="utf-8"

Content-Length: nnnn

SOAPAction: "Some-URI"

<SOAP-ENV:Envelope

xmlns:SOAP-ENV="<http://schemas.xmlsoap.org/soap/envelope/>"

```

SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<m:GetLastTradePrice xmlns:m="Some-URI">
<symbol>DIS</symbol>
</m:GetLastTradePrice>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

รูปที่ 2-8 ตัวอย่างของโซบริเควส

โซบเรสสปอนจะเป็นดังนี้

```

HTTP/1.1 200 OK
Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn

<SOAP-ENV:Envelope

xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
SOAP-encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<m:GetLastTradePriceREsponse xmlns:m="Some-URI">
<Price>34.5</Price>
</m:GetLastTradePriceREsponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

รูปที่ 2-9 ตัวอย่างของโซบเรสสปอน

นอกจากนั้นในโซบแมสเชจจะมีการใช้ เอ็กซ์เอ็มแอลเนมสเปซ ทุกๆ เอลเลอเมนต์ในเอกสารจะขึ้นต้นด้วยเนมสเปซซึ่งเนมสเปซจะถูกกำหนดโดยใช้เอ็กซ์เอ็มแอลเอนเอสเอทริบิว (xmlns attribute) ดังนี้

SOAP-Envelope

xmlns:SOAP-ENV="<http://schemas.xmlsoap.org/soap/envelope/>"

รูปที่ 2-10 เอ็กซ์เอ็มแอลเอนเอสเอทริบิว

แต่หากไม่ต้องการให้ขึ้นต้นด้วยเนมสเปซทำได้ดังนี้

<Envelope

xmlns="<http://schemas.xmlsoap.org/soap/envelope/>">

รูปที่ 2-11 เอ็กซ์เอ็มแอลเอนเอสเอทริบิวที่ไม่ขึ้นด้วยเนมสเปซ

2.3.1.1 เอนวิล๊อฟ (Envelope)

เอนวิล๊อฟเอนเอสเอทริบิวเป็นเอนเอสเอทริบิวที่บนสุดของเอกสารเอ็กซ์เอ็มแอลที่ใช้แสดงแมสเชจอาจประกอบด้วยเอทริบิว คือ เอนวิล๊อฟเนมสเปซ และ เอนโค้ดดิ้งสไตล์ (EncodingStyle)

- เอนวิล๊อฟเนมสเปซ (Envelop Namespace)– โซบแมสเชจจะแสดงเวอร์ชันโดยใช้เนมสเปซของเอนวิล๊อฟเอนเอสเอทริบิว (Element Namespace) จะถูกนำไปใช้ขึ้นต้นเอนวิล๊อฟ, เฮดเคอร์ และ บอดีเอนเอสเอทริบิว (Body Element)
- เอนโค้ดดิ้งสไตล์เอทริบิว (EncodingStyle Attribute) – ใช้แสดงวิธีซีริไรเซชัน (Serialization) ของ โซบแมสเชจ เอทริบิวนีสามารถมีได้ในทุกเอนเอสเอทริบิวและจะมีผลกับคอนเทนต์ (Content) ของเอนเอสเอทริบิวนั้นและ ภายเอนเอสเอทริบิวทั้งหมดของมันที่ได้ประกาศเอทริบิว

2.3.1.2 บอดี (Body)

เป็นส่วนของข้อมูลที่ใช้ในการแลกเปลี่ยน โดยทั่วไปข้อมูลจะเป็นการมาร์แชลอาร์พีซีคอล (Marshall RPC Call) และเออร์เรอร์รีพอร์ทภายเอนเอสเอทริบิว (Error Report Child Element) ของบอดีเอนเอสเอทริบิวที่จะถูกเรียกว่าบอดีเอนทรี (Body Entry)

บอดีเอนทรีจะต้องเป็นไปตามกฎนี้

- บอดีเอนทรีจะต้องแสดงด้วยชื่อเต็มประกอบด้วยเนมสเปซยูอาร์ไอ (Namespace URI) และ ชื่อของมัน (Local Name)
- โซบแมสเชจเอทริบิว (SOAP EncodingStyle Attribute) อาจจะมีได้ เพื่อแสดงถึงวิธี เอนโค้ด (Encode) ของบอดีเอนทรีนั้น

2.3.1.3 เฮดเดอร์ (Header)

เป็นส่วนเพิ่มเติมของเมสเสจสามารถประกอบด้วยอินฟอร์เมชันที่ระบุไปยังแอปพลิเคชัน ส่วนเพิ่มเติมนี้อาจนำไปใช้อิมพลิเมนต์เป็นอเพนทีเคชัน (Authentication), ทรานแซกชันแมนเนจเมนต์ (Transaction Management) เป็นต้น ทุก ๆ ซาเบิลโคมেন্টของเฮดเดอร์โคมেন্ট จะถูกเรียกว่า เฮดเดอร์เอนทรี (Header Entry)

เฮดเดอร์เอนทรีจะต้องเป็นไปตามกฎดังนี้

- เฮดเดอร์เอนทรีจะต้องแสดงด้วยชื่อเต็มประกอบด้วยเนมสเปซยูอาร์ไอ และ ชื่อของมัน
- อาจมีโชมเอนโค้ดดิ้งสไคล์แอททริบิวต์ที่ใช้สำหรับเฮดเดอร์เอนทรี
- โชมมัสอันเดอร์สแตนแอททริบิวต์ (SOAP mustUnderStand Attribute) และ โชมแอคเตอร์
- แอททริบิวต์ (SOAP actor attribute) จะมีหรือไม่มีก็ได้ เพื่อใช้บอกว่าจะจัดการกับเอนทรี นั้น อย่างไรและโดยใคร

```
<soap:Header>
  <m:local xmlns:m="http://www.w3schools.com/local/"
    soap:actor="http://www.w3schools.com/appml/">
    <m:language>en</m:language>
    <m:currency>USD</m:currency>
  </m:local>
</soap:Header>
```

รูปที่ 2-12 ตัวอย่างของโชมเฮดเดอร์

2.3.1.3.1 โชมแอคเตอร์แอททริบิวต์

โชมเมสเสจสามารถถูกส่งไปยังปลายทางผ่านตัวกลางของโชม (SOAP Intermediary) ตัวกลางของโชม คือ แอปพลิเคชันที่มีความสามารถทั้งรับและส่งต่อโชมเมสเสจ ในการส่งต่อตัวกลางจะต้องไม่ส่งต่อเฮดเดอร์โคมেন্টไปให้กับแอปพลิเคชัน โดยตัวกลางนี้จะถูกกำหนดเป็นยูอาร์ไอ ถ้าไม่มีแอททริบิวต์นี้หมายถึงผู้รับโชมเมสเสจเป็นปลายทางสุดท้าย

2.3.1.3.2 โชมมัสอันเดอร์สแตนแอททริบิวต์

ใช้แสดงว่า เฮดเดอร์เอนทรีนี้จำเป็นหรือเป็นเพียงออปชันสำหรับผู้รับที่จะนำไปประมวลผลค่าของมันคือ “1” หรือ “0” ถ้าไม่มีการระบุจะมีค่าเท่ากับ “0” โดย “0” หมายถึงเป็นออปชัน และ “1” หมายถึงจำเป็น โดยจะต้องประมวลผลให้ถูกต้องตามซีเมนติก (Semantic) หรือ ต้องไม่เพว (Fail)

เช่น แมสเซจเป็นส่วนหนึ่งของทรานแซกชัน ถ้าปลายทางไม่รองรับทรานแซกชัน จะต้องไม่ประมวลผลและส่งผลผิดพลาดกลับไปในกรณีที่มันส์อันเดอร์สแตนแอททริบิว เป็น “1” แต่ถ้าเป็น “0” จะยังคงประมวลผลต่อไป

2.3.2 โซบฟอลเอลเอเลเมนต์ (SOAP Fault Element)

ข้อความแสดงความผิดพลาดจากแอปพลิเคชันของโซบจะเก็บอยู่ในฟอลเอลเอเลเมนต์ ซึ่งถ้ามีจะต้องปรากฏในบอดีเอลเอเลเมนต์เพียงครั้งเดียวในโซบแมสเซจ ตัวอย่างอาจเป็นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:MustUnderstand</faultcode>
      <faultstring>Mandatory Header error.</faultstring>
      <faultactor>http://www.wrox.com/heroes/endpoint.asp</faultactor>
      <detail>
        <w:source xmlns:w="http://www.wrox.com/">
          <module>endpoint.asp</module>
          <line>203</line>
        </w:source>
      </detail>
    </soap:Fault>
  </soap:Body>
```

รูปที่ 2-13 ตัวอย่างของโซบแมสเซจที่มีความผิดพลาด

โชนฟอลเอลเลอเม้นท์มีเอลเลอเม้นท์ย่อย ๆ ดังตารางนี้

Sub Element	Description
<faultcode>	โค้ดที่ระบุถึงการ error
<faultstring>	ข้อความการ error
<faultactor>	ใครเป็นสาเหตุของการ error
<detail>	ระบุอินฟอร์เมชันของการ error

ตารางที่ 2-1 ตัวอย่างของเอลเลอเม้นท์ย่อยๆ

ค่าของฟอล โค้ด (Fault Code) สามารถมีค่าได้ดังตารางนี้

Error	Description
VersionMismatch	namespace ภายใน SOAP Envelope element ไม่ถูกต้อง
MustUnderstand	child element ของ Header element กับ mustUnderstand attribute ที่มีค่า “1” ผู้รับไม่รับรอง
Client	แมสเชจที่รูปแบบไม่ถูกต้อง หรือมีอินฟอร์เมชันไม่ถูกต้อง
Server	เกิดปัญหากับเซิร์ฟเวอร์ ไม่สามารถประมวลผลได้

ตารางที่ 2-2 ตัวอย่างของค่าฟอล โค้ด

2.3.3 โชนเอนโค้ดดิ้ง (SOAP Encoding)

โชนเอน โค้ดดิ้ง มีวิธีการแมพ (Map) จากไทป์ของโปรแกรมมิงไปเป็นเอ็กซ์เอ็มแอล 2 วิธี คือจากภายนอกโดยใช้ดับเบิลวีสดีแอล ที่บอกถึงไทป์ของข้อมูลที่รับ หรือ ส่ง หรือใช้เอ็กซ์เอ็มไอไทป์แอททริบิวต์ (xsi:type Attribute) ในกรณีที่ใช้ภาษาที่ใช้ไม่รองรับ ดับเบิลวีสดีแอล โดยทั้ง 2 วิธีจะใช้เอ็กซ์เอ็มแอลสคริปต์มาในการระบุไทป์ ตัวอย่าง ของการใช้เอ็กซ์เอ็มไอจะเป็นดังนี้

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>
    <m:MixedMessage xmlns:m="http://www.wrox.com/mix/">
      <param1 xsi:type="xsd:string">OU812</param1>
      <param2 xsi:type="xsd:integer">2001</param2>
      <param3 xsi:type="xsd:double">3.14159</param3>
    </m:MixedMessage>
  </soap:Body>
</soap:Envelope>

```

รูปที่ 2-14 ตัวอย่างของการใช้เอนโวกซ์เอสแอล

โชนั้นรองรับไทพ์ของข้อมูลได้ทั้งซิมเปิ้ลไทพ์ (Simple Type) และ คอมพาวไทพ์ (Compound Type) เช่น สตริก (Struct) และ อะเรย์ (Array)

2.3.3.1 สตริก

สตริกบุ๊ก (Struct Book) ที่เขียนด้วยภาษาซีพลัสพลัส (C++) จะเป็นดังนี้

```

Struct Book
{
    string author;
    string name;
    int page;
};

```

รูปที่ 2-15 ตัวอย่างของสตริกบุ๊ก

โซบแมสเซจที่ไม่ใช้ดับบิวเอสดีแอล แต่ใช้เอ็กซ์เอสไอ จะเป็นดังนี้

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:searchBook xmlns:ns1="http://soapinterop.org" SOAP-
      ENV:encodingStyle="http://schemas.xmlsoap.org/soap/envelope/">
      <bookResult xmlns:ns2="http://soapinterop.org/xsd" xsi:type="ns:Book">
        <author xsi:type="xsd:string">Ed Roman</author>
        <name xsi:type="xsd:string">Masterring EJB</name>
        <page xsi:type="xsd:int">500</page>
      </bookResult>
    </ns1:searchBook>
  </SOAP-ENV:Body>
</SOAP_ENV:Envelope>
```

รูปที่ 2-16 ตัวอย่างของโซบแมสเซจที่ใช้เอ็กซ์เอสไอ

แต่ถ้าใช้ดับบิวเอสดีแอล ในส่วนของการประกาศสคริปมา จะต้องประกอบด้วยสคริปมา ดังนี้

```
<types>
  <schema targetNamespace="http://soapinterop.org/xsd"
    xmlns="http://www.w3.org/2001/XMLSchema"
    xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
    xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
    <elementFormDefault="qualified">
      <complexType name="Book">
        <sequence>
          <element name="auhor" type="string"/>
          <element name="name" type="string"/>
          <element name="page" type="int"/>
        </sequence>
      </complexType>
    </elementFormDefault>
  </schema>
</types>
```

```

        </sequence>
    </complexType>
</schema>
</type>

```

รูปที่ 2-17 ตัวอย่างของโชนแมสเซจที่ใช้ดับบิวเอสดีแอล

2.3.3.2 อะเรย์

ตัวอย่างนี้เป็นอะเรย์ของสตริงบุ๊กในตัวอย่างก่อน โชนแมสเซจที่ไม่ใช่ดับบิวเอสดีแอล แต่ใช้ เอ็กซ์เอสไอจะเป็นดังนี้

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:BookArrayResult xmlns:ns1="http://soapinterop.org/" SOAP-ENV:encodingStyle
      ="http://schemas.xmlsoap.org/soap/encoding/">
      <BookArray xmlns:ns2="http://schemas.xmlsoap.org/soap/encoding/" xsi:type="ns2:Array"
        xmlns:ns3="http://soapinterop.org/xsd" ns2:arrayType=ns3:Book[3]>
        <item xsi:type="ns3:Book">
          <author xsi:type="xsd:string">Ed Roman</author>
          <name xsi:type="xsd:string">Mastering EJB</name>
          <page xsi:type="xsd:int">500</page>
        </item>
        <item xsi:type="ns3:Book">
          <author xsi:type="xsd:string">Roger Session</author>
          <name xsi:type="xsd:string">The battle of the middleware</name>
          <page xsi:type="xsd:int">350</page>
        </item>
        <item xsi:type="ns3:Book">
          <author xsi:type="xsd:string">Chris Dix</author>
          <name xsi:type="xsd:string">Professional XML Web Service</name>
          <page xsi:type="xsd:int">550</page>
        </item>
      </BookArray>
    </ns1:BookArrayResult>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

```

</item>

</BookArray>

</ns1:BookArrayResult>

<SOAP-ENV:Body>

<SOAP-ENV:Envelope>

```

รูปที่ 2-18 ตัวอย่างของอะเรย์ของสตริงที่ใช้อีเอชเอสไอ

แต่ถ้าใช้ ดับเบิลเอนคอดในส่วนของการประกาศสคริปต์จะต้องประกอบด้วยสคริปต์ดังนี้

```

<types>

  <schema targetNamespace="http://soapinterop.org/xsd"

    xmlns="http://www.w3.org/2001/XMLSchema"

    xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding">

    xmlns:wsdl="http://schemas.xmlsoap.org/wsdl"

    <elementFormDefault="qualified">

      <complexType name="Book">

        <sequence>

          <element name="author" type="string"/>

          <element name="name" type="string"/>

          <element name="page" type="int"/>

        </sequence>

      </complexType>

      <complexType name="ArrayOfBook">

        <complexContent>

          <restriction base="SOAP-ENC:Array">

            <attribute ref="SOAP-ENC:Array"/>

          </restriction>

        </complexContent>

      </complexType>

    </schema>

  </type>

```

รูปที่ 2-19 ตัวอย่างของอะเรย์ของสตริงที่ใช้ดับเบิลเอนคอด

2.3.4 โขบในเอชทีทีพี

ในการส่งโอบผ่านทางเอชทีทีพีจะต้องใช้คอนเทนไทป์ (Content-Type) เป็นเท็กเอ็ชเอ็มแอล (Text/xml) แต่ในโอบรีเควสจะต้องมี เฮดเคอร์โอบแอคชั่น (Header SOAPAction) ภายในเอชทีทีพีเฮดเคอร์ (HTTP Header) ตัวโอบแอคชั่น (SOAPAction) จะเป็นตัวบอกให้เซิร์ฟเวอร์รู้ว่าเอชทีทีพีโพส (HTTP Post) นั้นเป็นโอบแมสเชจ และค่าของเฮดเคอร์ คือ ยูอาร์ไอที่แสดงถึงจุดหมายของโอบแมสเชจส่วนโอบเรสสปอน จะต้องมีสเตตัสโค้ด (Status Code) ตามมาตรฐานของเอชทีทีพี โดย 200 - 299 แสดงว่าสำเร็จ แต่ถ้าเรสสปอนแมสเชจเป็นการฟอล (Fault) แล้วสเตตัสโค้ดจะต้องเป็น 500 ซึ่งแสดงถึง อินเทอร์นัลเซิร์ฟเวอร์เออเรอ (Internal Server Error) ตัวอย่าง ของเรสสปอนที่มีสเตตัสโค้ดเป็น 500 อาจเป็นดังนี้

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml
Content-Length: ###
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:VersionMismatch</faultcode>
      <faultstring>The SOAP namespace is incorrect.</faultstring>
      <faultactor>http://www.wrox.com/endpoint.asp</faultactor>
      <detail>
        <w:errorinfo xmlns:w="http://www.wrox.com/">
          <desc>The SOAP namespace was blank.</desc>
        </w:errorinfo>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

รูปที่ 2-20 ตัวอย่างของเรสสปอนที่มีสเตตัสโค้ดเป็น 500

2.3.5 โขบสำหรับอาร์พีซี

จุดประสงค์ของการออกแบบโขบ คือ ทำอาร์พีซีโดยใช้เอ็กซ์เอ็มแอลการเรียกอาร์พีซีนั้นจะแมพเข้ากับเอชทีทีพีรีเควส และ อาร์พีซีเรสสปอน (RPC Response) จะแมพกับเอชทีทีพีเรสสปอน

ในการทำเมธอดคอล (Method Call) จำเป็นที่จะต้องมียีนฟอร์เมชันดังนี้

2.3.5.1 ยูอาร์ไอ ของออบเจกต์ปลายทาง (Target Object)

- ชื่อของเมธอด
 - คำอธิบายของเมธอด (Method Signature) ส่วนนี้เป็นออปชัน
 - พารามิเตอร์สำหรับเมธอด
 - ข้อมูลส่วนหัว (Header Data) เป็นออปชัน
- พิจารณาเมธอดดังนี้

```
double GetStockQuote ( [in] string sSymbol );
```

รูปที่ 2-21 ตัวอย่างของเมธอด

ถ้าเนมสเปซของเมธอดคือ “http://www.worxstox.com/” แล้วการเรียกเมธอดโดยรีเควสสต็อกโควท (Request Stock Quote) ใช้ซิมโบโลยูแปดหนึ่งสอง (Symbol OU812) จะเป็นดังนี้

```
<q:GetStockQuote xmlns:q="http://www.worxstox.com/">
  <q:symbol xsi:type="xsd:string">OU812</q:symbol>
</q:GetStockQuote>
```

รูปที่ 2-22 ตัวอย่างของโขบเมสเสจที่เรียกเมธอดโดยรีเควสสต็อกโควท

ชื่อเมธอดและชื่อของเอลเลอเมนต์จะต้องตรงกัน (Match) เช่นเดียวกับพารามิเตอร์สำหรับเรสสปอน จะต้องชื่อเอลเลอเมนต์ จะต้องเป็นชื่อเมธอดตามด้วยเรสสปอน ดังนี้

```
<q:GetStockQuoteResponse xmlns:q="http://www.worxstox.com/">
  <q:ret xsi:type="xsd:double">100.0</q:ret>
</q:GetStockQuoteResponse>
```

รูปที่ 2-23 ตัวอย่างของชื่อเมธอดตามด้วยเรสสปอน

ชื่อเอเลเมนต์ของค่าที่คืนมาสามารถเปลี่ยนได้ โดยจะกำหนดได้จากโปรแกรมที่เขียน หรือจากคัมบิวเอสดีแอล

2.3.6 โซบทุลคิต (SOAP Toolkit)

โซบทุลคิต คือเครื่องมือที่จะทำหน้าที่ในการประมวลผลโซบรีเควส และส่งเรสสปอนกลับไปที่ไคลเอนต์โดยโซบทุลคิต แต่ละตัวใช้ได้บนแพลตฟอร์มที่ต่างกัน รองรับเทคโนโลยีเว็บเซอร์วิสที่ต่างกัน บางตัวอาจจะรองรับคัมบิวเอสดีแอล หรือ ยูคียูไอ ในขณะที่บางตัวรองรับอย่างใดอย่างหนึ่งหรือไม่รองรับทั้งสองอย่าง และถึงแม้ว่าโซบจะเป็นอิสระต่อแพลตฟอร์ม แต่ในโซบทุลคิตบางตัวยังไม่สามารถทำงานข้ามผลิตภัณฑ์ (Interoperability) ได้ เนื่องจากรองรับเทคโนโลยีของเว็บเซอร์วิสไม่เหมือนกัน เช่น เอ็กซ์เอสไอในบางผลิตภัณฑ์จะบังคับให้โซบแมสเชจจะต้องระบุไทป์ด้วยเอ็กซ์เอสไอ หากไม่มีก็จะเป็นฟอล (Fault) ซึ่งในปัจจุบันแต่ละผลิตภัณฑ์ก็พยายามพัฒนาให้สามารถทำงานด้วยกันได้

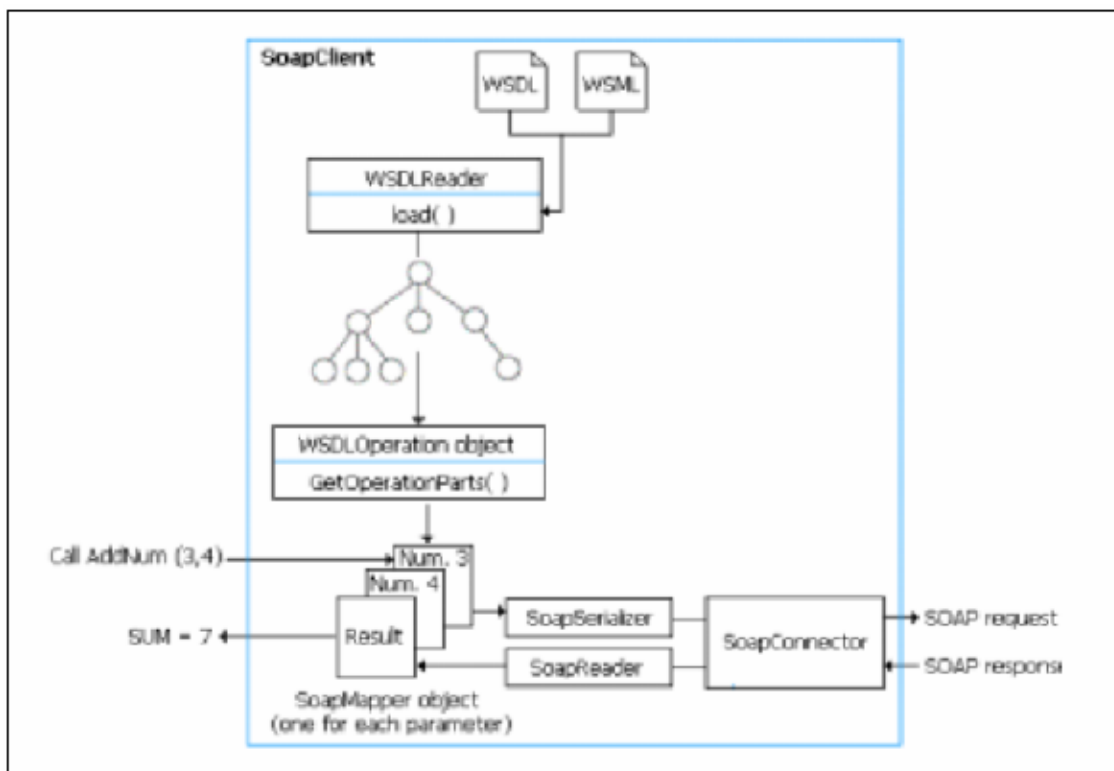
ชื่อ	แพลตฟอร์ม	แหล่งที่มา
4s4c	COM	http://www.4s4c.com
A SOAP for RPC NT Service	COM	http://www.whitemesa.com
Apache Axis	Java	http://xml.apache.org/axis
Apache SOAP	Java	http://xml.apache.org/soap
CapeConnect	Java	http://www.capeclear.com
Glue	Java	http://www.themindelectric.com
IBM Web Service Toolkit	Java	http://www.alphaworks.ibm.com
IdooXoop for Java and C++	Java, C++	http://www.zvon.org
Microsoft SOAP Toolkit	COM	http://www.microsoft.com
PacketSOAP	COM	http://www.packetsoap.com
SOAP::Lite	Perl	http://www.soaplite.com
Visual Studio .NET	COM	http://www.microsoft.com

ตารางที่ 2-3 ตัวอย่างของโซบทุลคิต

แบ่งแยกช่องทางที่ใช้ในการส่งโซบแมสเซจ

2.3.7.2 การประมวลผลภายในโซบไคลเอนต์ออบเจกต์

มีการทำงานดังรูป



รูปที่ 2-25 ตัวอย่างของการทำงานการประมวลผลภายในโซบไคลเอนต์ออบเจกต์

ดับเอสดีแอลรีดเดอร์ออบเจกต์ (WSDL Reader Object) จะทำการโหลดไฟล์ดับบิวเอสดีแอล และไฟล์ดับบิวเอสเอ็มแอลเข้าไปไว้ในคอม (DOM) แล้วทำการวิเคราะห์โดยดับบิวเอสดีแอลรีดเดอร์ออบเจกต์ (WSDLReader Object) จะสร้างดับบิวเอสแอลโอเปอเรชั่นออบเจกต์ (WSDLOperation object) เพื่อทำการเรียกใช้งานออปเปอร์เรชั่น (ตามรูปนี้คือ AddNum(3, 4)) หลังจากนั้นดับบิวเอสแอลโอเปอเรชั่นจะเรียกเมธอด GetOperationParts เพื่อดึงรูปแบบแมสเซจของอินพุตและเอาท์พุทของออปเปอร์เรชั่นออกมาแล้วโซบไคลเอนต์ (SoapClient) จะสร้างโซบแมพเปอร์ออบเจกต์ (Soap Mapper Objects) ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้มาจากการเรียกใช้ออปเปอร์เรชั่น เข้าไว้ในออบเจกต์เหล่านี้ พอถึงขั้นตอนนี้ โซบซีริไรส์ออบเจกต์ (SoapSerializer Objects) จะสร้างโซบรีเควสแมสเซจ (SOAP Request Message) จาก โซบแมพเปอร์ออบเจกต์ (SoapMapper Objects) ที่เหมาะสมและส่งไปที่เซิร์ฟเวอร์

เมื่อเซิร์ฟเวอร์ประมวลผลเสร็จแล้วก็จะส่งโซบเรสพอนกลับไปที่ไคลเอนต์ โซบไคลเอนต์ออบเจกต์จะทำการโหลดผลลัพธ์ของออปเปอร์เรชั่นไปเก็บในโซบแมพเปอร์ออบเจกต์ที่เหมาะสมแล้ว

ส่งผลลัพธ์กลับไปให้แอปพลิเคชันของผู้ใช้

2.3.7.3 การสร้างโซบเซิร์ฟเวอร์ออบเจกต์ (SoapServer Object)

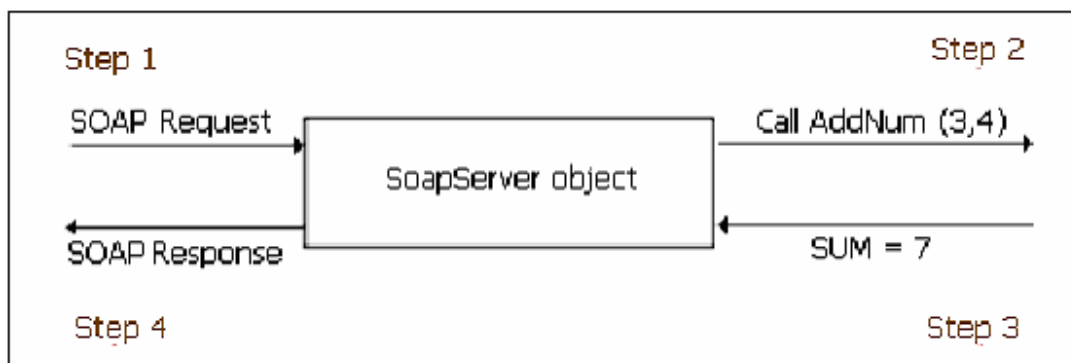
ส่วนทางฝั่งเซิร์ฟเวอร์เราสามารถกำหนดรูปแบบการรับโซบรีเควส (SOAP Listener) ได้ 2 รูปแบบ คือ อินเทอร์เน็ตเซิร์ฟเวอร์เอพีไอเซิร์ฟเวอร์ (Internet Server API Server) หรือ ไอเอสเอพีไอเซิร์ฟเวอร์ (ISAPI Server) และ แอกทีฟเซิร์ฟเวอร์เพจเซิร์ฟเวอร์ (Active Server Pages Server) หรือเอเอสพีเซิร์ฟเวอร์ (ASP Server)

```
<definitions>
...
<service name="DocSample1">
  <port name="DocSample1PortType" binding="tns:DocSample1Binding">
    <soap:address location="http://localhost/ DocSample1Test/ DocSample1.wsdl"/>
  </port>
</service>
...
</definitions>
```

รูปที่ 2-26 ตัวอย่างของโซบเซิร์ฟเวอร์ออบเจกต์

2.3.7.4 การทำงานทางฝั่งเซิร์ฟเวอร์

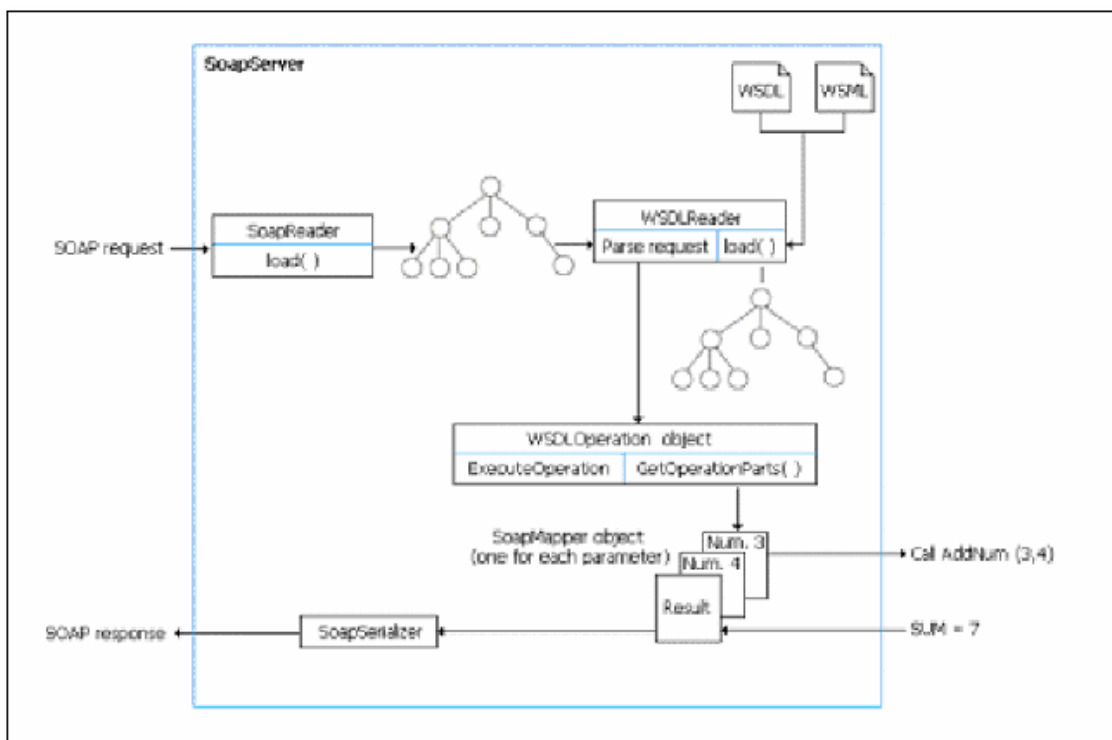
ไม่ว่าโซบรีเควสจะเรียกไปยังไอเอสเอพีไอเซิร์ฟเวอร์ หรือ เอเอสพีเซิร์ฟเวอร์ลิสเทนเนอร์ก็ตาม ข้อมูลที่รับส่งก็ยังคงเหมือนกัน เมื่อโซบเซิร์ฟเวอร์ออบเจกต์ ได้รับโซบรีเควสจากไคลเอนต์ ก็จะทำการแปลและเรียกไปยังเมธอด COM ที่ตรงกับออปเปอร์เรชันที่ต้องการ (ตามไคอะแกรมนี้คือเมธอด AddNum) หลังจากนั้นเมธอดที่ถูกเรียกใช้งานก็จะส่งผลลัพธ์กลับมาให้โซบเซิร์ฟเวอร์ออบเจกต์ซึ่งมันก็จะแปลงผลลัพธ์ให้อยู่ในรูปแบบของ โซบเรสสปอนแล้วจึงส่งกลับไปที่ไคลเอนต์ ดังรูป (Server – Side Data Flow)



รูปที่ 2-27 ตัวอย่างเซิร์ฟเวอร์ใช้ตัวคำไฟล์

2.3.7.5 การประมวลผลภายใน โซบเซิร์ฟเวอร์ออบเจกต์

เมื่อได้รับโซบริเวสจากไคลเอนต์แล้วโซบริดเดอร์ออบเจกต์ (SoapReader Object) ก็จะโหลด รีเวสแมสเชงไปไว้ คอมสตรักเตอร์ (DOM Structure) หนึ่ง ในขณะที่ดับบิวเอสดีแอลรีดเดอร์ (WSDLReader) จะโหลดไฟล์ ดับบิวเอสดีแอล และไฟล์ดับบิวเอสเอ็มแอลไปไว้ในอีก คอมสตรักเตอร์หนึ่ง ตัว ดับบิวเอสดีแอลรีดเดอร์จะแปลรีเวสและสร้างดับบิวเอสดีแอลออปเปอร์เรชั่นออบเจกต์ ขึ้นเพื่อใช้ใน รีเวสออปเปอร์เรชั่นหลังจากนั้น ดับบิวเอสดีแอลออปเปอร์เรชั่นออบเจกต์ จะเรียกใช้เมธอด GetOperationParts เพื่อดึงรูปแบบแมสเชงของอินพุตและเอาท์พุทของออปเปอร์เรชั่น ออกมา แล้วโซบเซิร์ฟเวอร์ จะสร้างโซบแมเพอร์ออบเจกต์ ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้มาจากการเรียกใช้ ออปเปอร์เรชั่น เข้าไว้ในออบเจกต์ เหล่านี้ พอถึงขั้นตอนนี้โซบเซิร์ฟเวอร์ออบเจกต์ จะทำการเรียกไปที่ คอมเมธอด (COM method) ที่ตรงกับที่ร้องขอเข้ามา ดังรูป



รูปที่ 2-28 ตัวอย่างของการทำงานการประมวลผลภายในโซบเซอร์ฟเวอร์ออบเจกต์

2.4 ดับบิวเอสดีแอล (WSDL)

2.4.1 ดับบิวเอสดีแอลคืออะไร

ดับบิวเอสดีแอล หรือ เว็บเซอร์วิสเดสคริปต์ชั้นแลงเกวจ (Web Service Description Language) นั้นถูกคิดค้นขึ้นโดยบริษัทไอบีเอ็ม (IBM) และบริษัทไมโครซอฟต์ (Microsoft) ซึ่งเป็นภาษาที่ใช้อธิบายคุณลักษณะการให้บริการของเว็บเซอร์วิส และวิธีการติดต่อขอรับบริการจากเว็บเซอร์วิส เช่น ชื่อเว็บเซอร์วิส, ชื่อเมธอดของคอมโพเนนท์ที่เปิดให้บริการ, พารามิเตอร์ (Parameter) ที่ส่งไปยังเมธอด, ชนิดข้อมูลของพารามิเตอร์ เป็นต้น โดยรายละเอียดเหล่านี้จะเป็นไปตามไวยากรณ์ของภาษาเอ็กซ์เอ็มแอล

2.4.2 ความสัมพันธ์ระหว่างดับบิวเอสดีแอลกับเว็บเซอร์วิส

โซบ หรือ ซิมเบิลออบเจกต์แอคเซสโพรโตคอล, ยูดีดีไอ หรือ ยูนิเวอร์ซัลเดสคริปต์ชั้นดิสคัฟเวอรีแอนดอินทิเกรชัน และ ดับบิวเอสดีแอล หรือ เว็บเซอร์วิสเดสคริปต์ชั้นแลงเกวจ ต่างก็เป็นแกนหลักสำคัญของ เว็บเซอร์วิสทั้งสิ้น สิ่งสำคัญในการเรียกใช้งานเว็บเซอร์วิส คือ ต้องรู้ที่อยู่ (จากยูดีดีไอ) รู้คุณลักษณะของ เว็บเซอร์วิสนั้นๆ จากเมธอดของเว็บเซอร์วิส และ พารามิเตอร์ต่างๆ ที่จำเป็น (จากเอกสารดับบิวเอสดีแอล) และร้องขอบริการ รวผลลัพธ์จากผู้ให้บริการ (ด้วยโซบโพรโตคอล)

ความสัมพันธ์ระหว่าง 3 ส่วน แสดงได้ดังนี้

- โขบทจัดเป็น โปรโตคอลสื่อสารที่อาศัยไวยากรณ์ของภาษาเอ็กซ์เอ็มแอล และทำงานกับโปรโตคอลอื่นๆ ได้หลายชนิด เนื่องจากใช้ไวยากรณ์เอ็กซ์เอ็มแอล จึงทำให้ทำงานได้ทุกแพลตฟอร์ม ดังนั้นเราก็เลยสามารถเรียกใช้คอมโพเนนท์ข้ามแพลตฟอร์มได้
- ดัชนีบิวเอสดีแอลทำหน้าที่ที่เรียกว่า เซอร์วิสเดสคริปชัน (Service Description) คือเป็นเอกสารที่ใช้อธิบายคุณลักษณะของเว็บเซอร์วิสที่ได้กล่าวมาแล้ว โดยใช้ไวยากรณ์ของเอ็กซ์เอ็มแอล เช่นกัน
- ยูคียูไอโอเปรียบเสมือนฐานข้อมูลเก็บรายละเอียดของเว็บเซอร์วิสไว้ และรอให้ผู้มาใช้บริการมาค้นหาบริการ บทบาทนี้เรียกว่า เซอร์วิสดีคัฟเวอรี (Service Discovery) ส่วนในกรณีของผู้ให้บริการ ก็ต้องนำข้อมูลเกี่ยวกับเว็บเซอร์วิสของตนไปเก็บไว้ในยูคียูไอโอบทบาทนี้ของยูคียูไอโอ คือ เซอร์วิสพับลิเคชัน (Service Publication)

2.4.3 โครงสร้างของดัชนีบิวเอสดีแอล

โครงสร้างของเอกสารดัชนีบิวเอสดีแอลนั้น แบ่งออกเป็น 2 ส่วนใหญ่ๆด้วยกัน คือ

2.4.3.1 แอ็บสแตร็กเคฟนิชันกรุป (Abstract Definitions Group)

ทำหน้าที่ คือ จะกำหนด เมสเสจ โขบทที่ไม่ขึ้นกับแพลตฟอร์ม หรือรูปแบบภาษา กล่าวคือไม่ได้บรรจุส่วนประกอบที่มีเฉพาะในเครื่องบางเครื่อง หรือภาษาบางภาษา โดยที่ส่วนประกอบของแอ็บสแตร็กนั้น จะประกอบไปด้วย ไทป์ (type), เมสเสจ (Message) และพอร์ตไทป์ (portType)

- ไทป์ ประกอบด้วยส่วนประกอบสคริปต์ที่กำหนดรูปแบบข้อมูลต่างๆ โดยที่ข้อมูลเหล่านั้นไม่ยึดติดแบบรูปแบบ แพลตฟอร์ม และภาษา
- เมสเสจ ทำหน้าที่อธิบายข้อมูลที่ใช้ในการแลกเปลี่ยนระหว่างผู้ให้บริการเว็บเซอร์วิส กับผู้ขอใช้บริการ โดยที่เว็บเมธอด (Web Method) แต่ละตัวจะมี 2 เมสเสจ คือ อินพุต กับ เอาท์พุต โดยที่อินพุตนั้น ทำหน้าที่อธิบายพารามิเตอร์สำหรับเว็บเมธอด ส่วนเอาท์พุตจะอธิบายข้อมูลที่เว็บเมธอดนั้นส่งกลับมา ซึ่งเมสเสจแต่ละตัวนั้นอาจจะไม่มีหรือมีการระบุพารามิเตอร์ของเว็บเมธอด โดยผ่านทางพารามิเตอร์ <พอร์ต> (parameter <port>) ซึ่งพารามิเตอร์แต่ละตัว จะเกี่ยวข้องกับรูปแบบของข้อมูลคอนกรีตที่กำหนดไว้ในส่วนประกอบคอนเทนเนอร์ <ไทป์> (Container <types>)
- พอร์ตไทป์ ประกอบไปด้วยเซต (Set)ของ แอ็บสแตร็กโอเปอเรชัน (Abstract Operations) ซึ่งตัวโอเปอเรชันนี้ ก็จะอ้างอิงถึงอินพุตเมสเสจ (Input Message) และ เอาท์พุตเมสเสจ (Output Message)

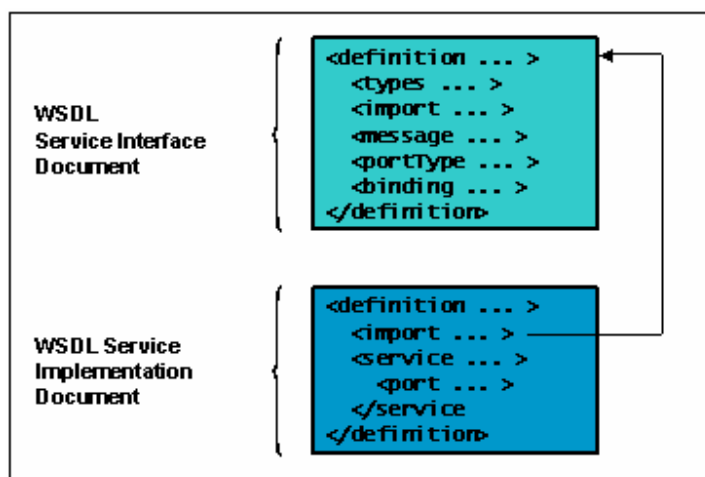
2.4.3.2 คอนกรีตเดสคริปชันกรุป (Concrete Descriptions Group)

จะทำหน้าที่ตรงกันข้ามกับส่วนของแอ็บสแตร็กชัน (Abstract Section) คือจะเป็นข้อมูลที่ขึ้นอยู่กับขนาด (Size), เครื่องและภาษา โดยประกอบด้วย บายคิง (Binding) และ เซอร์วิส (Service)

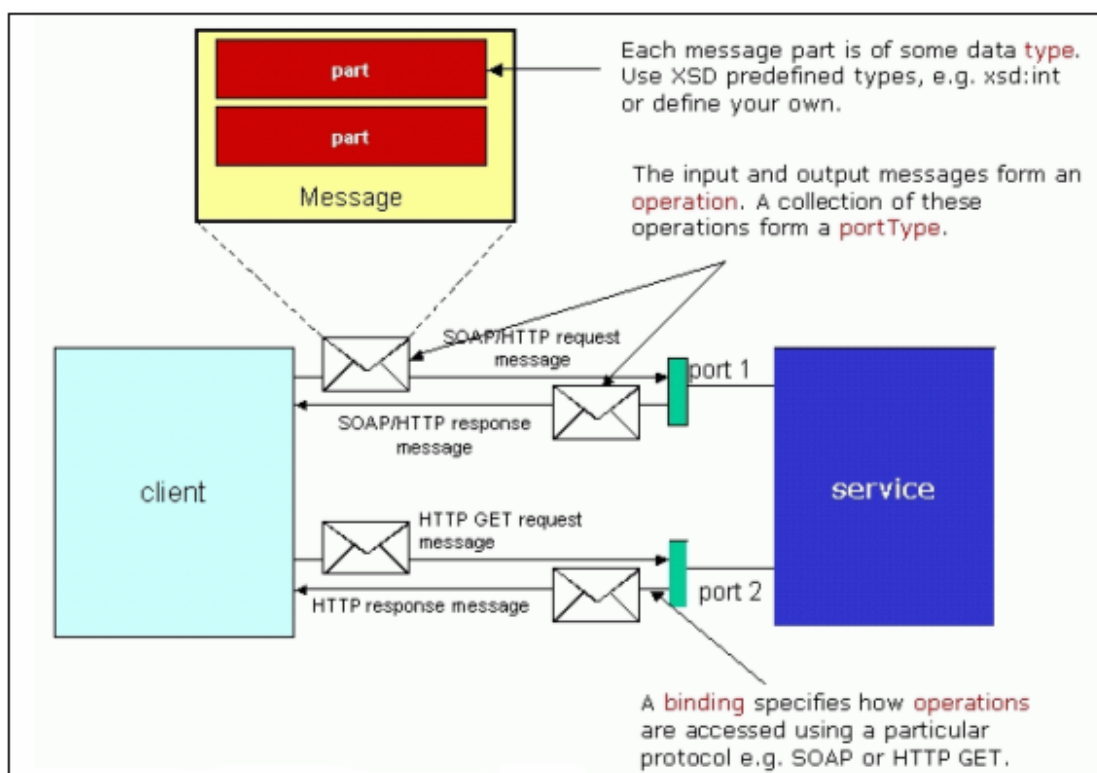
- **บายคิง** ทำหน้าที่อธิบายโปรโตคอลโดยที่บายคิงนั้นประกอบไปด้วยออปเปอร์เรชั่น, อินพุตแมสเสจ โดยที่ส่วนประกอบของบายคิงนั้นจะเหมือนกันกับพอร์ตไทป์
- **เซอร์วิส** ประกอบไปด้วยคอลเล็กชัน (Collection) ของพอร์ตต่างๆ โดยที่แต่ละพอร์ตนั้นจะจับพอร์ตโพรดัก (Support Product) ที่ต่างกัน

นอกจากนั้นแล้วยังมีส่วนอื่นๆอีกด้วย

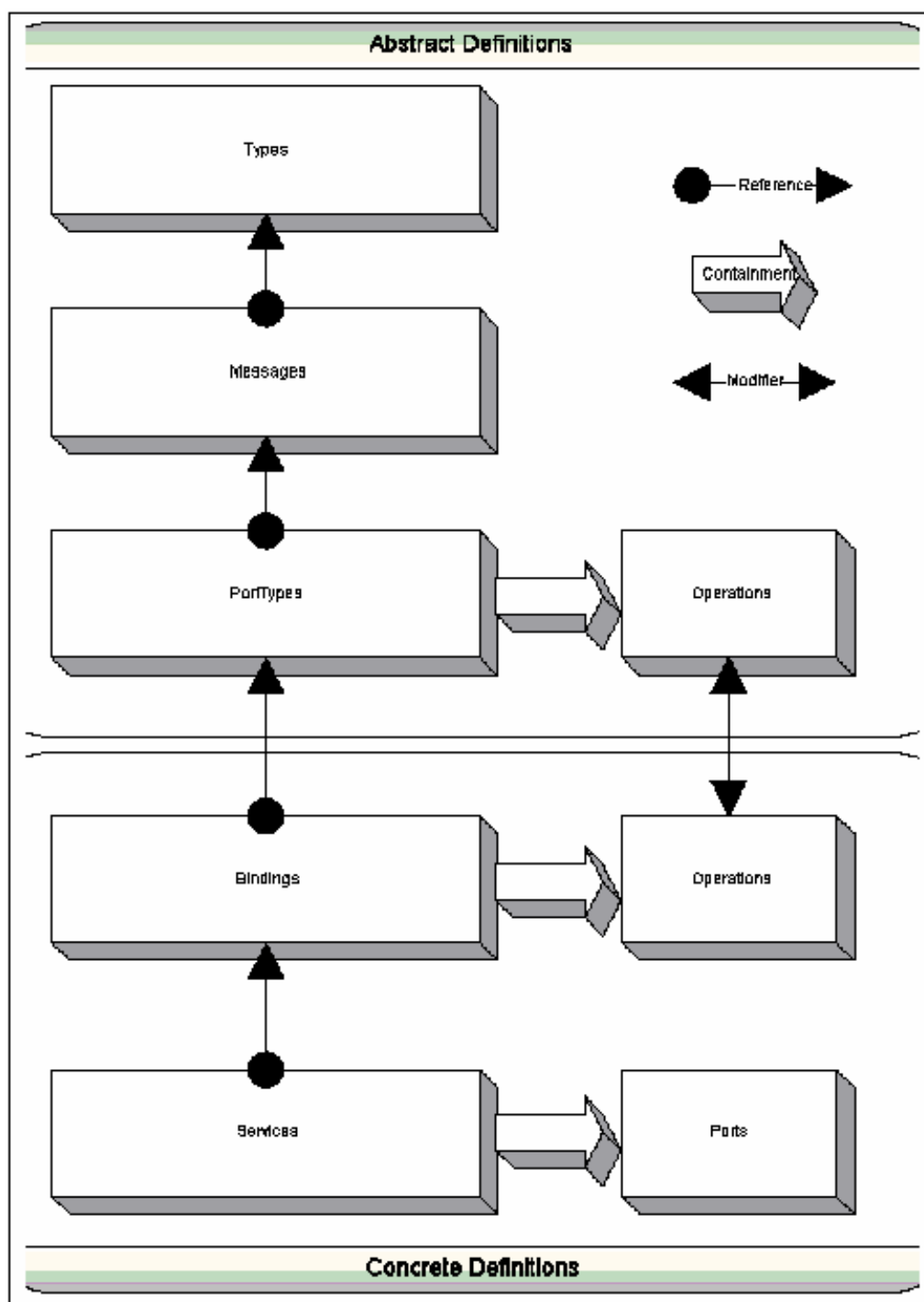
- **พอร์ต** จะระบุแอดเดรส (Address) ของบายคิง แล้วทำการดีไฟน์ซิงเกิลคอมมูนิเคชันเอนด์พอยท์ (Define Single Communication Endpoint)
- **ออปเปอร์เรชั่น** (Operation) จะทำการรวมพอร์ตต่างๆที่เกี่ยวข้องนั้นเข้าด้วยกัน



รูปที่ 2-29 ตัวอย่างของโครงสร้างของเอกสารฉบับวีเอสดีแอล



รูปที่ 2-30 แสดงการส่งเอกสารฉบับวีเอสดีแอล



รูปที่ 2-31 แสดงส่วนประกอบของคัมบิวเอสดีแอล

```

<?xml version = "1.0"?>
<definitions name = "StockQuoteService" targetNamespace =
  "http://www.xmethods.net/sd/StockQuoteService.wsdl"
  xmlns:tns = "http://www.xmethods.net/sd/StockQuoteService.wsdl"
  xmlns:xsd = "http://www.w3.org/1999/XMLSchema"
  xmlns:soap = "http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns = "http://schemas.xmlsoap.org/wsdl/">
  <message name = "getQuoteRequest">                                // จุดสังเกตที่ 3
    <part name = "xsd:symbol" type = ":string"/>
  </message>
  <message name = "getQuoteResponse">                                // จุดสังเกตที่ 4
    <part name = "Result" type = "xsd:float"/>
  </message>
  <portType name = "StockQuotePortType">
    <operation name = "getQuote">
      <input message = "tns:getQuoteResponse" name = "getQuoteResponse"/>
    </operation>
  </portType>
  <binding name = "getQuoteBinding" type = "tns:getQuotePortType">
    <soap:binding style = "rpc" transport = "http://schemas.xmlsoap.org/soap/http/">
      <operation name = "getQuote">                                // จุดสังเกตที่ 2
        <soap:operation soapAction="">
          <input>
            <soap:body use = "encode" namespace = "urn:xmethods-delayed-quotes"
              encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/">
          </input>
          <output>
            <soap:body use = "encode" namespace = "urn:xmethods-delayed-quotes"
              encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/">
          </output>
        </operation>

```

```

</binding>
<service name = "StockQuoteService">                                // จุดสังเกตที่ 1
    <documentation>Obtains 20 minute delayed quotes from Yahoo</documentation>
    <port name = "StockQuotePort" binding = "tns:StockQuoteBinding">
        <soap:address location = "http://service.xmethods.net:80/soap/">    // จุดสังเกตที่ 5
    </port>
</service>
</definitions>

```

รูปที่ 2-32 เอกสารฉบับวีเอสดีแอล

ในเอกสารฉบับวีเอสดีแอล แต่ละไฟล์ มี 5 จุด ที่ควรรู้ ดังที่แสดงข้างต้น โดยเรียงลำดับตามความสำคัญจากมากไปน้อยดังนี้

จุดสังเกตที่ 1 บอกชื่อเว็บเซอร์วิส

สำหรับเว็บเซิร์ฟเวอร์หนึ่งๆ จะมีเว็บเซอร์วิส จำนวนกี่บริการก็ได้ และชื่อเว็บเซอร์วิส ก็เป็นตัวจำแนกและบอกแต่ละบริการซึ่งห้ามซ้ำกัน โดยตำแหน่งในเอกสารฉบับวีเอสดีแอลที่บอกชื่อเว็บเซอร์วิส คือในแอตทริบิวเนม (Attribute Name) ของ เอลิเมนต์เซิร์ฟวิส (Element <service>) สำหรับชื่อ เว็บเซอร์วิส StockQuoteService

จุดสังเกตที่ 2 บอกชื่อเมธอดของเว็บเซอร์วิส

สำหรับเว็บเซอร์วิสหนึ่งๆ จะมีเมธอดจำนวนกี่เมธอดก็ได้ ยกตัวอย่าง กรณีเว็บเซอร์วิสที่ให้บริการเครื่องคิดเลข อาจจะมีเมธอดแอด (Method Add (บวก)), ซับแทรซ (Subtract (ลบ)), มัลติพลาย (Multiply (คูณ)), ดีไวด์ (Divide (หาร)) และอื่นๆ โดยตำแหน่งในเอกสารฉบับวีเอสดีแอลที่บ่งบอกชื่อเมธอดอยู่ที่แอตทริบิวเนมของเอลิเมนต์โอเปอเรชัน (Element <operation>) ซึ่งชื่อเมธอดของเว็บเซอร์วิสในตัวอย่างนี้ คือ getQuote

จุดสังเกตที่ 3 และ 4 บอกข้อมูลพารามิเตอร์ของเมธอด ของเว็บเซอร์วิส และค่าที่ส่งกลับมา

ปกติแล้วในเอกสารฉบับวีเอสดีแอลจะมีเอลิเมนต์เมสเสจ (Element <message>) เป็นคู่ๆ ซึ่งก็คือข้อมูลพารามิเตอร์ของ เมธอดที่ส่งไปให้เว็บเซอร์วิสประมวลผล และข้อมูลผลลัพธ์ที่ส่งกลับมา ส่วนใหญ่แล้วเราจะดูที่จุดสังเกตที่ 3 และ 4 เพื่อให้ทราบชนิดข้อมูล ว่าพารามิเตอร์ที่ส่งไปและข้อมูลที่ส่งกลับเป็นชนิดอะไร สำหรับตัวอย่างนี้ข้อมูลพารามิเตอร์เป็นชนิด สตริง (String เพราะเมธอด getQuote ให้ใส่พารามิเตอร์ เป็นชื่อย่อของบริษัทที่ต้องการทราบราคาหุ้น) ส่วนข้อมูลที่ส่งกลับเป็นชนิดโฟลต (Float ตัวเลขดัชนีราคาหุ้นซึ่งเป็นเลขทศนิยม)

จุดสังเกตที่ 5 บอกตำแหน่งของไฟล์ซึ่งทำหน้าที่เป็นโซบลิสเทนเนอร์ (SOAP Listener)

ดังที่เคยกล่าวไว้แล้วว่าไฟล์โซบลิสเทนเนอร์ (File SOAP Listener) จะทำหน้าที่คอยฟังว่ามีการเรียกเว็บเซอร์วิสหรือไม่ ถ้ามีก็จะไปปลุกให้เว็บเซอร์วิสนั้นๆทำงาน และ ตำแหน่งของไฟล์นี้ก็บอกด้วยแอททริบิวต์ `location` ใน เอเลเมนต์ `<address>`

ตามทฤษฎีแล้วไฟล์เอกสารดับเบิลยูเอสดีแอล แต่ละไฟล์สามารถอธิบายคุณลักษณะของบริการเว็บเซอร์วิสได้มากกว่า 1 บริการ โดยแต่ละเว็บเซอร์วิสจะมีพอร์ตสื่อสารเฉพาะตัว ซึ่งบ่งบอกไว้ในเอกสารดับเบิลยูเอสดีแอล อยู่แล้ว

2.5 ยูดีดีไอ (UDDI)

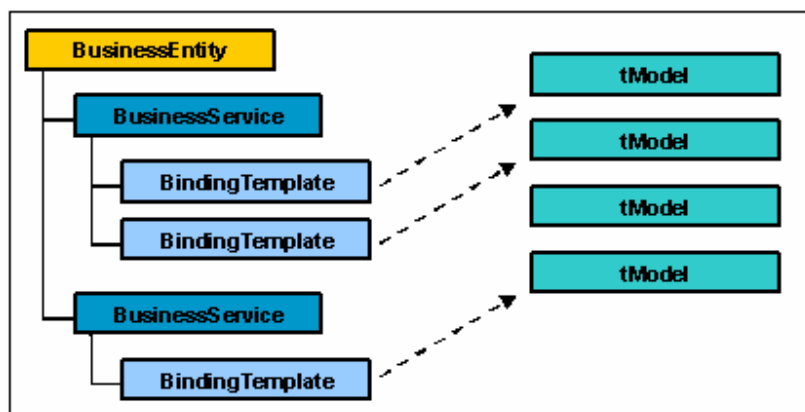
2.5.1 ยูดีดีไอคืออะไร

ยูดีดีไอ หรือ ยูนิเวอร์ซัลเดสคริปชันคิสก์พเวอร์แอนคอินทิเกรชัน (Universal Description Discovery and Integration) คือเป็นตัวที่ให้เมธอด สำหรับการสร้างและการหารายละเอียดของเซอร์วิส โดยที่ ยูดีดีไอดาต้าเอนทิตี (UDDI Data Entities) จะเป็นตัวกำหนดทั้งข้อมูลบิสซิเนส (Business Data) และข้อมูลเซอร์วิส (Service Data) ที่ถูกกำหนดในดับเบิลยูเอสดีแอลจะถูกคอมพลีท (Complete) ไปไว้ในยูดีดีไอรีจิสทรี (UDDI Registry) ซึ่ง ยูดีดีไอนั้นจะสนับสนุนเซอร์วิสได้หลายๆประเภท ดังนั้น ยูดีดีไอนั้นจะไม่ขึ้นกับดับเบิลยูเอสดีแอล

2.5.2 โครงสร้างของยูดีดีไอ

จะมีประเภทของข้อมูลอยู่ 4 ตัวด้วยกัน เรียกว่าไพรมารีดาต้าไทป์ (Primary Data Types) ในดาต้ารีจิสทรี

1. บิสซิเนสเอนทิตี (Business Entity)
2. บิสซิเนสเซอร์วิส (Business Service)
3. บายคิงเทมเพลต (Binding Template)
4. ทีโมเดล (tModel)



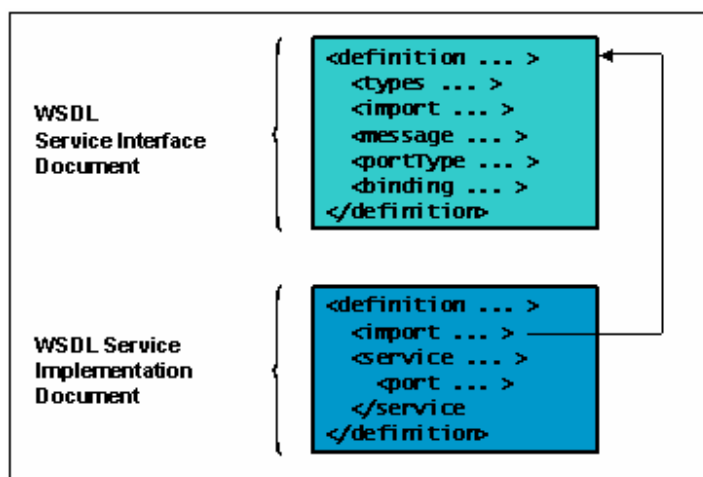
รูปที่ 2-33 โครงสร้างของยูคิไอ

บิสซิเนสเอนทิตีนั้นจะให้ข้อมูลเกี่ยวกับบิสซิเนส และ บรรจุบิสซิเนสเซอร์วิสหนึ่งอัน หรือมากกว่านั้น ถ้าเป็น บิสซิเนสเซอร์วิสนั้น เทคนิคและวิธีการอธิบายลักษณะของบิสซิเนส สำหรับเว็บเซอร์วิสจะถูกกำหนด ลงใน บิสซิเนสเซอร์วิส และ บายคิงเทมเพลต ซึ่งแต่ละ บายคิงเทมเพลตจะบรรจุทีโมเดลหนึ่งอัน หรือมากกว่านั้น โดยที่แต่ละทีโมเดล จะใช้ในการกำหนดรายละเอียดทางเทคนิค (Technical Specification) สำหรับแต่ละเซอร์วิส

2.5.3 ประเภทของเอกสารดับบิวเอสดีแอล

เอกสารดับบิวเอสดีแอลที่จะทำการประกาศในยูคิไอ มี 2 ประเภทคือ

- เซอร์วิสอินเทอร์เฟซ (Service Interfaces)
- เซอร์วิสอิมพลีเม้นท์เทชัน (Service Implementations)



รูปที่ 2-34 ประเภทของเอกสารดับบิวเอสดีแอล

เซอร์วิสอินเทอร์เฟซนั้นถูกอธิบายโดยเอกสารฉบับวีสดีแอลที่ประกอบไปด้วยไพล์, อิมพอร์ต, แมสเชจ และ บายคิง ส่วน เซอร์วิสอินเทอร์เฟซ จะประกอบด้วย ฉบับวีสดีแอลเซอร์วิสเคฟิเนชัน (WSDL service definition) ซึ่งจะใช้ในการอิมพลิเมนต์ 1 เซอร์วิส หรือ มากกว่านั้น

เซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์ (Service Interface Document) นั้นสามารถอ้างอิง เซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์ตัวอื่น ได้ในการอิมพอร์ตเอลเลอเม้นท์ (Import Element) โดยเซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์ที่ถูกอิมพลิเมนต์แล้ว จะมีอิมพอร์ต และ เซอร์วิสเอลเลอเม้นท์ซึ่งเซอร์วิสคือคิวกิเมนต์ ที่อิมพลิเมนต์เหล่านั้นแล้วจะบรรจุเคสคริปต์ชันของเซอร์วิสที่อิมพลิเมนต์ตัวเซอร์วิสอินเทอร์เฟซแล้ว

อิมพอร์ตเอลเลอเม้นท์อย่างน้อย 1 ตัว หรือมากกว่านั้น จะบรรจุเรฟเฟอเรนส์ (Reference) ของ ฉบับวีสดีแอลเซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์ (WSDL Service Interface Document) ได้มากกว่า 1 อัน

อิมพอร์ตเอลเลอเม้นท์ (Import Element) ใน ฉบับวีสดีแอลเซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์ นั้น จะบรรจุ 2 แอททริบิวต์ โดยที่ แอททริบิวต์เนมสเปซ (Attribute Namespace) จะเก็บยูอาร์แอล ที่ตรงกับ targetNamespace ใน เซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์

ส่วนแอททริบิวต์โลเคชัน (Attribute Location) จะเก็บยูอาร์แอลที่ใช้เรฟเฟอเรนส์ฉบับวีสดีแอลคือคิวกิเมนต์ที่บรรจุคอมพลีทเซอร์วิสอินเทอร์เฟซเคฟิเนชัน (Complete Service Interface Definition)

ส่วนแอททริบิวต์บายคิง (Attribute Binding) บน พอร์ตจะเรฟเฟอเรนส์ถึงสเปซิฟิค (Specific Binding) ในเซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์

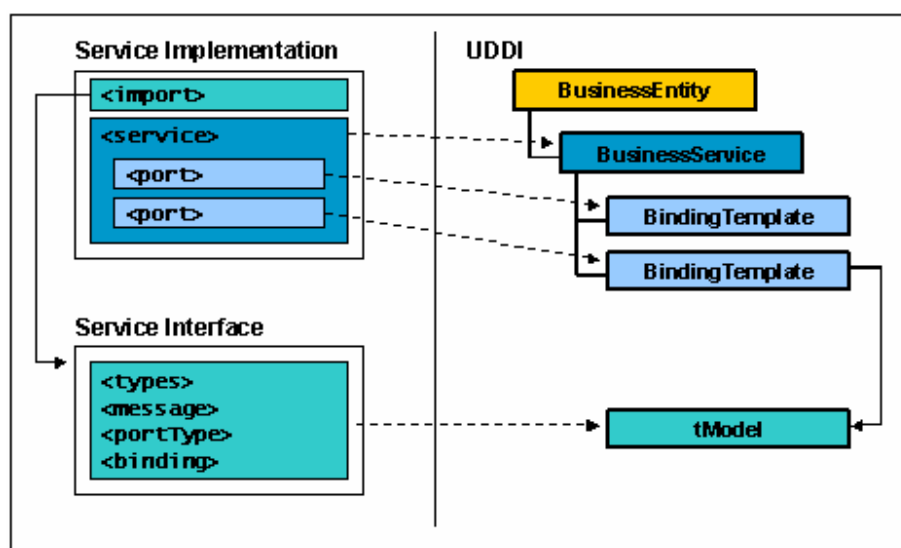
- เซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์ ถูกพัฒนาและประกาศโดย เซอร์วิสอินเทอร์เฟซโพรไวเดอร์ (Service Interface Provider)
- เซอร์วิสอินเทอร์เฟซคือคิวกิเมนต์ ถูกพัฒนาและประกาศโดย เซอร์วิสโพรไวเดอร์ (Service Provider)

2.5.4 การประกาศและค้นหาเอกสารฉบับวีสดีแอลในยูดีดีไอ

เซอร์วิสอินเทอร์เฟซ (Service Interface) เมื่อรวมกับ เซอร์วิสอิมพลิเมนต์เทชั่นคือคิวกิเมนต์ (Service Implementation Document) ก็จะกลายเป็น เอกสารฉบับวีสดีแอลที่สมบูรณ์แล้ว

เมื่อเซอร์วิสอินเทอร์เฟซนั้นทำการกำหนดเซอร์วิส (Define Service) มันจะต้องทำการประกาศ เซอร์วิส (Published Service) ลงในยูดีดีไอริจิสทรีเหมือนกับที่โมเดล หลังจากนั้นเซอร์วิสอิมพลิเมนต์เทชั่นจะอธิบายอินสแตนส์ (Describe Instances) ของเซอร์วิส ซึ่งแต่ละอินสแตนส์ (Instance) ก็จะทำการกำหนดการใช้เอกสารฉบับวีสดีแอลเซอร์วิสเอลเลอเม้นท์ (WSDL Service Element) ซึ่งเซอร์วิสเอลเลอเม้นท์ (Service Element) จะถูกเผยแพร่ในยูดีดีไอบิสซิเนสเซอร์วิส

เมื่อเอกสารฉบับวีสดีแอลเซอร์วิสเคสคริปต์ชันนั้นถูกประกาศ (Publishing) แล้ว ในส่วนของ เซอร์วิสอินเทอร์เฟซก็จะต้องทำการประกาศ เหมือนกับส่วนของที่โมเดล ก่อนที่ในส่วนของเซอร์วิสอิมพลิเมนต์เทชั่นจะทำการประกาศเหมือนกับส่วนของบิสซิเนสเซอร์วิส หรือที่เรียกว่าการแมปปิง (Mapping)



รูปที่ 2-35 การประกาศและค้นหาเอกสารฉบับวิเอสซีแอลในยูดีดีไอ

2.6 คอม86 (COM86)

ชุดพัฒนาคอม86 เป็นชุดเครื่องมือสำหรับใช้ในการพัฒนาอุปกรณ์ทางด้านระบบฝังตัวที่อยู่บนพื้นฐานการพัฒนาโดยใช้แพลตฟอร์มที่มีไมโครโปรเซสเซอร์ตระกูล x86 ซึ่งเป็นที่คุ้นเคยกับผู้ใช้งานในปัจจุบันเป็นส่วนประกอบหลักในการทำงาน

2.6.1 จุดเด่นของคอม86

2.6.1.1 ใช้ไมโครคอนโทรลเลอร์ที่มีความสามารถสูง

ภายในชุดพัฒนาคอม86 มีบอร์ดคอม86 ซึ่งได้เลือกใช้ไมโครคอนโทรลเลอร์ Am186CC ของบริษัท เอเอ็มดี (AMD) ซึ่งเป็นไมโครคอนโทรลเลอร์ขนาด 16 บิตทำงานอยู่ที่ความเร็ว 24 เมกะเฮิร์ต (MHz) ทำให้มีความสามารถในการประมวลผลต่างๆเพิ่มมากขึ้นเมื่อเทียบกับการพัฒนาโดยใช้ไมโครคอนโทรลเลอร์ขนาด 8 บิตที่นิยมใช้กันอยู่ในปัจจุบัน นอกจากนี้ภายในตัวไมโครคอนโทรลเลอร์เองยังได้รวมเอาอุปกรณ์พื้นฐานต่างๆที่จำเป็นในการใช้งาน เช่น ดีเรียมคอมโทรลเลอร์ (DRAM controller), อินเทอร์รัพคอนโทรลเลอร์ (Interrupt controller), ยูอาร์ต (UART) เข้าไว้ในตัวไมโครคอนโทรลเลอร์ซึ่งทำให้ง่ายต่อการออกแบบอุปกรณ์เพื่อทำเป็นสินค้าภายหลังการพัฒนาตัวอย่างเช่น การนำบอร์ดคอม86 ไปพัฒนาเป็นอุปกรณ์อย่างหนึ่งที่ต้องการควบคุมการทำงานผ่านทางยูเอสบีพอร์ต (USB port) จะสามารถทำได้โดยง่ายเนื่องจากไม่ต้องการออกแบบวงจรควบคุมการทำงานในส่วนยูเอสบี นี้เพิ่มอีก เพียงแต่ออกแบบวงจรการทำงานส่วนอื่นที่ต้องการ

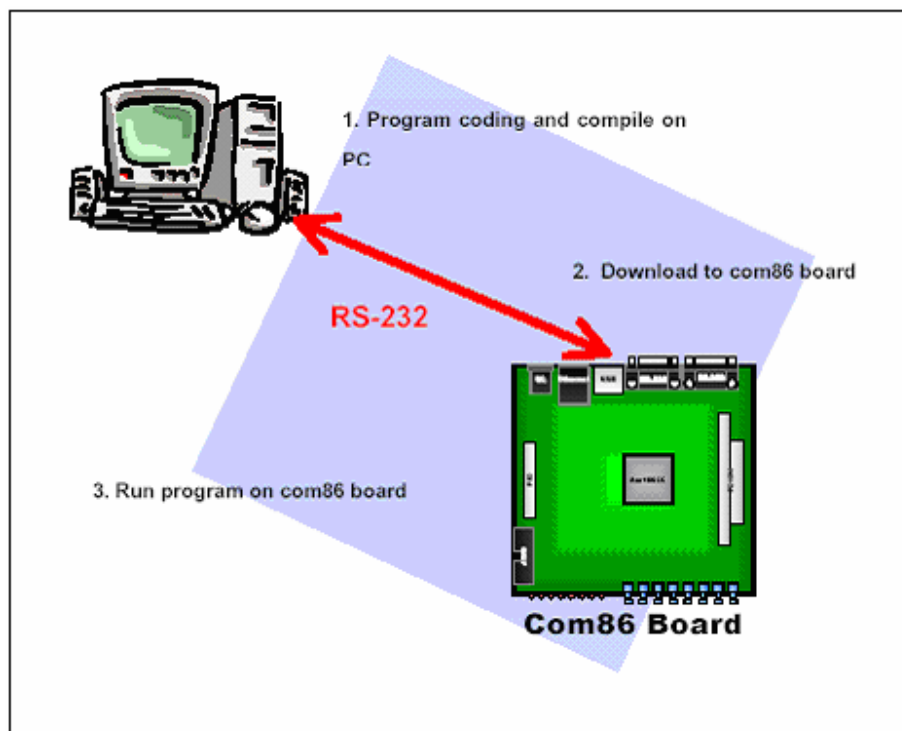
เพิ่มเติมทำให้ทุ่นเวลาในการพัฒนา เป็นต้น นอกจากนี้ภายในไมโครคอนโทรลเลอร์ยังเพิ่มส่วนโมดูล (Module) ทางด้านการสื่อสารต่างๆ เข้าไปภายในตัว ทำให้เป็นไมโครคอนโทรลเลอร์ที่มีความสามารถเด่นทางด้านการสื่อสารเหมาะกับการพัฒนาอุปกรณ์ที่ต้องใช้คุณสมบัติการสื่อสารต่างๆ ซึ่งจะสามารถทำได้ง่ายเมื่อเทียบกับการใช้ไมโครคอนโทรลเลอร์ขนาดเล็กแบบเดิมซึ่งมีข้อจำกัดต่างๆ อยู่มากมาย

2.6.1.2 มีเครื่องมือในการพัฒนาโปรแกรมมาก

เนื่องจากการพัฒนาโปรแกรมโดยใช้ชุดพัฒนาคอม86 นั้นเป็นเสมือนกับการพัฒนาโปรแกรมบนของคอมพิวเตอร์ส่วนบุคคลทั่วไป ดังนั้นจึงสามารถนำโปรแกรมเครื่องมือต่างๆ ที่ใช้พัฒนาโปรแกรมสำหรับคอมพิวเตอร์ส่วนบุคคลมาใช้งานได้มากมาย และ โปรแกรมเครื่องมือเหล่านี้ก็เป็นพื้นฐานของนักพัฒนาจึงทำให้สามารถพัฒนาอุปกรณ์ได้อย่างรวดเร็ว ลดต้นทุนในการพัฒนาต่างๆ ได้เป็นจำนวนมากนอกจากนี้ภายในชุดพัฒนายังมีชุดซอฟต์แวร์เครื่องมือต่างๆ ที่จำเป็นสำหรับช่วยในการพัฒนาโปรแกรมด้วยอีกทางหนึ่ง

2.6.1.3 ง่ายในการเรียนรู้และพัฒนา

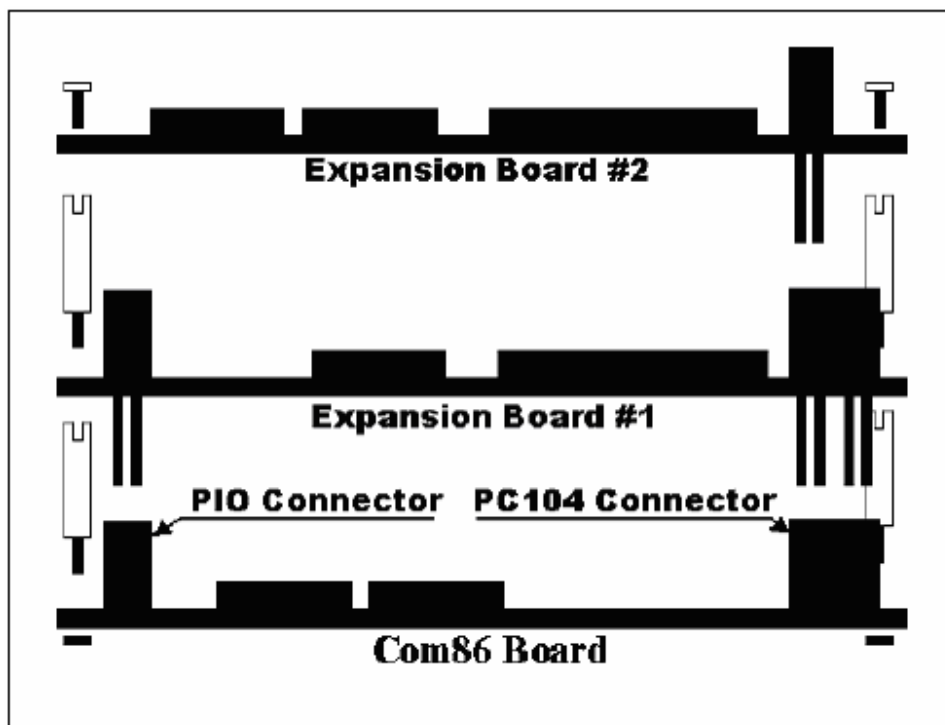
บอร์ดคอม86 ภายในชุดพัฒนาคอม86 นี้มีสถาปัตยกรรมของไมโครคอนโทรลเลอร์แบบ x86 ซึ่งเป็นสถาปัตยกรรมเดียวกับคอมพิวเตอร์ส่วนบุคคลทั่วไป โดยมีชุดคำสั่งที่เข้ากันได้กับชุดคำสั่งของ 80286 (Real mode) ทำให้ง่ายต่อการเรียนรู้ในการพัฒนาโปรแกรมต่างๆ และยังสามารถพัฒนาและทดสอบซอฟต์แวร์ต่างๆ ได้บนคอมพิวเตอร์ ก่อนที่จะทดสอบกับบอร์ดคอม86 ทำให้สามารถพัฒนาอุปกรณ์ทางด้านระบบฝังตัวต่างๆ ได้อย่างรวดเร็ว ซึ่งรูปแบบการพัฒนาที่มีลักษณะเป็นไปดังภาพ



รูปที่ 2-36 แสดงถึงรูปแบบการพัฒนาโปรแกรมโดยใช้บอร์ดคอม86

2.6.1.4 มีความสามารถในการเพิ่มขยายได้

ในส่วนของการเพิ่มขยายของฮาร์ดแวร์ (Hardware) สามารถทำได้โดยผ่านทางคอนเนคเตอร์ (Connector) ของบอร์ดในรูปแบบของมาตรฐาน (ซีพียูหนึ่งศูนย์สี่) PC/104 ซึ่งจะมีการเชื่อมต่อแบบบัส อินเทอร์เฟซ (BUS Interface) แบบไอเอสเอบัส (ISA BUS) ขนาด 8 บิต คล้ายกับไอเอสเอบัส ที่มีอยู่ใน คอมพิวเตอร์ส่วนบุคคลทั่วไป ซึ่งผู้พัฒนาสามารถออกแบบฮาร์ดแวร์ให้เป็นลักษณะของบอร์ดขยายและนำมา เชื่อมต่อกับบอร์ดคอม86 และเพียงเขียนซอฟต์แวร์ไดรเวอร์ (Software Driver) สำหรับควบคุมการทำงานของ ส่วนเพิ่มเข้ามาใหม่ซึ่งจะทำงานร่วมกับระบบปฏิบัติการในการรองรับการทำงานของแอปพลิเคชันต่างๆที่เพิ่ม เข้ามา นอกจากนี้ตัวบอร์ดคอม86 ยังสามารถเชื่อมต่อกับอุปกรณ์ภายนอกในลักษณะของพีไอโอ (PIO) หรือ โปรแกรมเมเบิลอินพุต/เอาต์พุต (Programmable Input/Output) เหมือนกันการใช้งานไมโครคอนโทรลเลอร์ รุ่นอื่นๆได้ด้วยเช่นกัน



รูปที่ 2-37 แสดงการเชื่อมต่อของบอร์ดขยายกับบอร์ดคอม86

2.6.2 แอปพลิเคชันเป้าหมายของชุดพัฒนา (Target Application)

จากความสามารถของไมโครคอนโทรลเลอร์ของบอร์ดคอม86 ทำให้การประยุกต์ใช้งานกับแอปพลิเคชันต่าง ๆ นั้น สามารถทำได้มากมาย โดยตัวอย่างแอปพลิเคชันต่างๆที่เหมาะสมแก่การพัฒนาโดยบอร์ดคอม86 มีดังนี้

2.6.2.1 เกี่ยวกับอุตสาหกรรม

ตัวอย่างของอุปกรณ์ควบคุมภายในโรงงาน เช่น การควบคุมเครื่องจักรจากระยะไกล การตรวจวัดสถานะต่างๆ ภายในโรงงานและส่งกลับไปยังศูนย์กลางควบคุมโดยการใช้บอร์ดคอม86 เป็นอุปกรณ์ที่ช่วยทำหน้าที่ในการสื่อสาร ควบคุมสถานะแวดล้อม หรือเครื่องจักรต่างๆภายในโรงงานและรายงานกลับไปยังศูนย์กลางตามกำหนดเวลา เป็นต้น

2.6.2.2 อุปกรณ์ยูเอสบี (USB Peripheral)

อุปกรณ์ยูเอสบีต่างๆ สามารถพัฒนาได้โดยการใช้บอร์ดคอม86 ซึ่งภายในบอร์ดคอม86 นี้มียูเอสบีคอนโทรลเลอร์ภายในตัวไมโครคอนโทรลเลอร์ Am186CC สามารถเชื่อมต่อเข้ากับคอมพิวเตอร์ได้ด้วยความเร็ว 12 เมกกะบิตเปอร์เซ็ค (Mbps) ซึ่งการพัฒนาเป็นอุปกรณ์ ยูเอสบี ต่างๆ โดยบอร์ดคอม86 นี้สามารถทำได้โดยไม่ต้องมีการต่อวงจรทางด้านยูเอสบีใดๆ เพิ่มเติมอีก

2.6.2.3 เว็บเซิร์ฟเวอร์ (Embedded web server)

เอ็มเบดเดดเว็บเซิร์ฟเวอร์ (Embedded web server) สามารถสร้างได้บนบอร์ดคอม86 โดยไม่จำเป็นต้องเชื่อมต่อวงจรอื่นๆ เพิ่มเติม ซึ่งบอร์ดคอม86 นี้สามารถสื่อสารกับภายนอกผ่านทางอีเทอร์เน็ตพอร์ตที่อยู่บนบอร์ดที่ความเร็ว 10 เมกกะบิตเปอร์เซ็ค ทำให้สามารถนำไปประยุกต์ใช้งานในกาออกแบบอุปกรณ์ควบคุมต่างๆ ผ่านเอ็มเบดเดดเว็บเซิร์ฟเวอร์ ที่ทำงานอยู่บนบอร์ดคอม86 ได้โดยง่าย

2.6.2.4 โรโบติก (Robotics)

บอร์ดคอม86 มีไมโครคอนโทรลเลอร์ที่มีประสิทธิภาพสูงในการทำงาน การพัฒนาซอฟต์แวร์สามารถพัฒนาได้ซับซ้อนมากยิ่งขึ้น การเชื่อมต่อขยายทางด้านฮาร์ดแวร์ต่างๆสามารถทำได้โดยง่าย เหมาะสำหรับการพัฒนาหุ่นยนต์ที่มีการทำงานที่ซับซ้อน

2.6.2.5 การควบคุมบ้านหรือสิ่งก่อสร้าง

อุปกรณ์สำหรับควบคุมระบบต่างๆภายในบ้าน หรืออาคารสามารถนำบอร์ดคอม86 ไปประยุกต์ใช้งานได้เช่นกัน โดยใช้เป็นส่วนในการควบคุมการทำงานของอุปกรณ์ต่างๆ หรือคอยตรวจสอบการทำงานของเซนเซอร์ (Sensor) ต่างๆ ภายในบ้าน และแจ้งไปยังเจ้าของบ้านเมื่อเกิดเหตุร้ายขึ้น

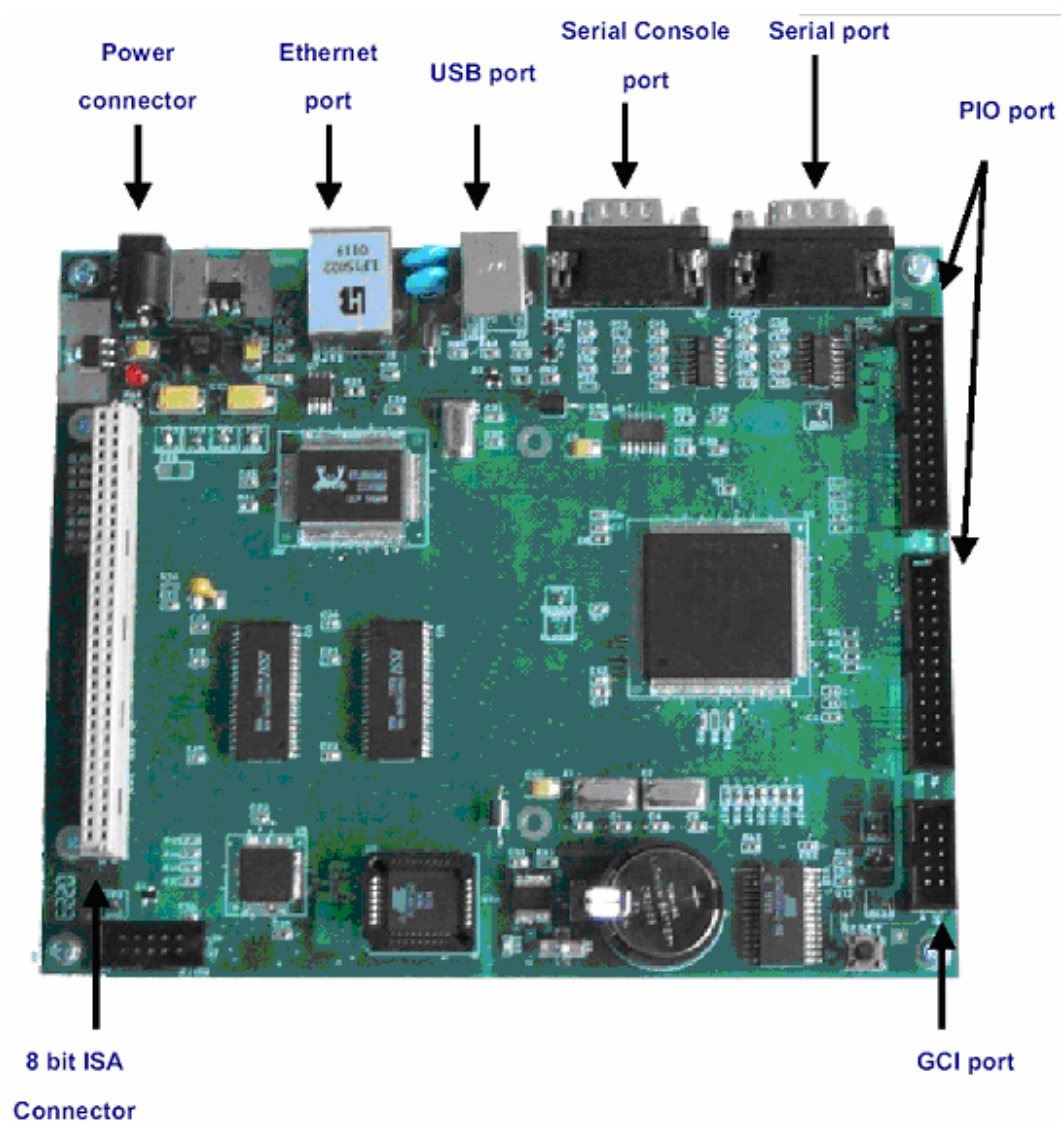
นอกจากนี้ตัวไมโครคอนโทรลเลอร์ยังรองรับการนำไปสร้างเป็นอุปกรณ์ต่างๆ ที่เกี่ยวข้องกับงานทางด้านการสื่อสารต่างๆเช่น ไอเอสดีเอ็น โมเด็มแอนด์เทอร์มินัลแอดาปเตอร์ (ISDN Modem and Terminal Adaptor), โลเอนด์เราเตอร์ (Low-end Router), ไลน์การ์ดแอปพลิเคชัน (Line card application), เอ็กซ์ดีเอส

แอลแอปพลิเคชัน (xDSL application), ดิจิตอลคอร์ดโฟน (Digital corded phone) เป็นต้น ซึ่งสามารถนำบอร์ดคอม86 ไปประยุกต์ใช้งานในการออกแบบพัฒนาได้เช่นกัน

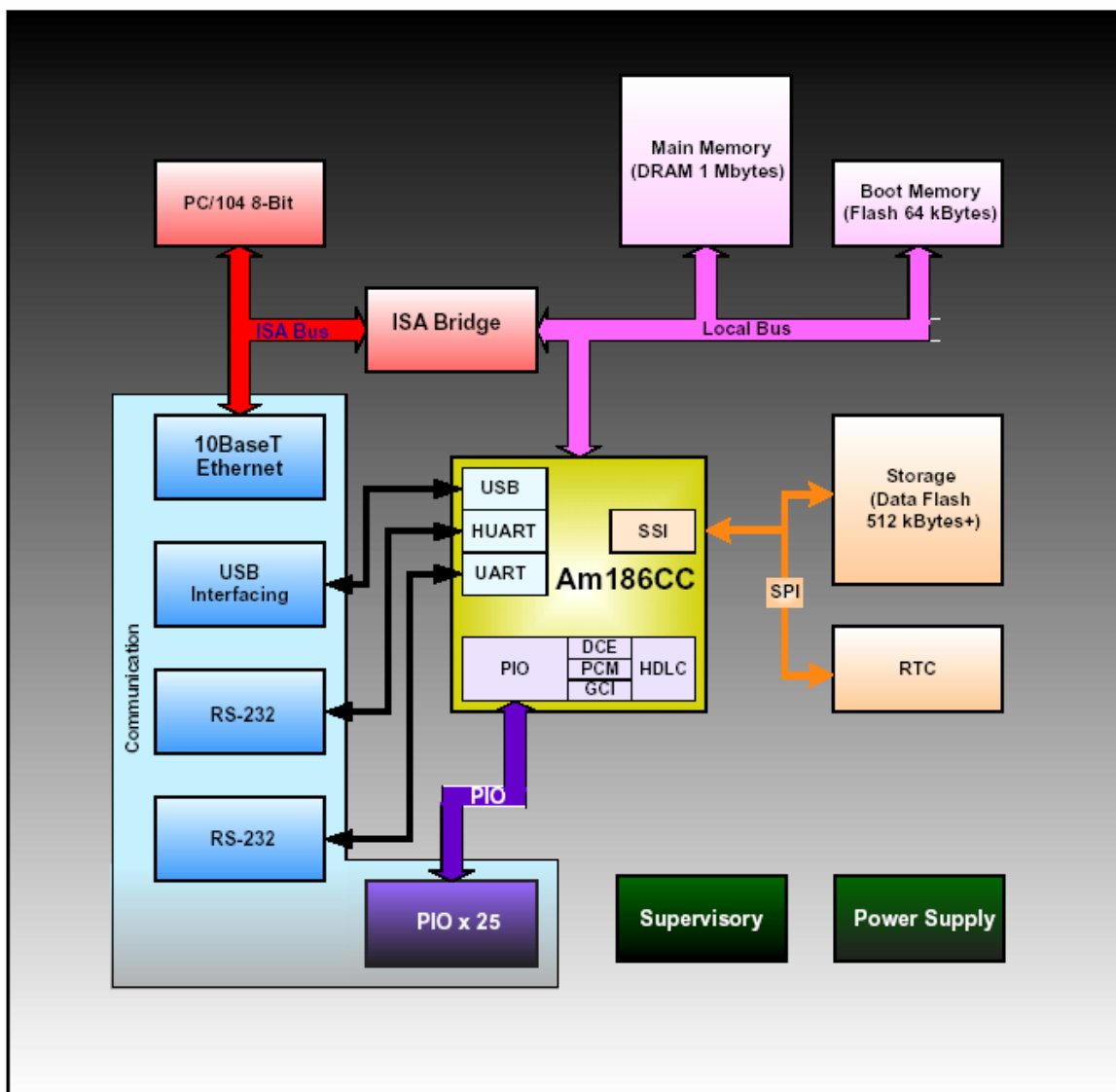
2.6.3 คุณสมบัติของบอร์ดคอม86

บอร์ดคอม86มีคุณสมบัติดังนี้

- ซีพียู เอซีหนึ่งแปดหกซีซี (CPU Am186CC)
- ดีแรม 1 เมกกะไบต์ (1 MBytes DRAM)
- แฟลชไบออส 64 กิโลไบต์ (64 kBytes Flash BIOS)
- 512 kBytes Serial DataFlash® storage (upgradeable)
- Full duplex IEEE 802.3 compliant on board Ethernet controller with RJ45 connector
- USB peripheral controller version 1.1 with on-board type B USB connector
- Dual RS232C-serial port with on-board DB-9 connector
- Four High level Data Link Control (HDLC) channels (multiplex with PIO)
- Four independent Time slot Assigner (TSAs)
- GCI Controller with on-board GCI port for external interface
- Three programmable 16-bit timers
- Real-Time clock
- Watchdog timer and reset controller
- Interrupt Controller (10 external sources)
- 25 programmable I/O
- 8 bit ISA bus in PC/104 form factor
- 3.3 V and 5 V on-board power supply
- LED Indicators for power supply and Ethernet



รูปที่ 2-38 ตัวอย่างของภาพชุดพัฒนาคอม86



รูปที่ 2-39 บล็อกไดอะแกรมแสดงส่วนประกอบต่างๆของคอม86